

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Mini-Project — Requirements Deliverable

Software Requirements Specification

for

Student Record Management System (SRMS)

Version 2.0

Prepared in accordance with IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018

Prepared by: Team 17 (Project ID SRMS-2026-17)

B Varun (PES1UG24CS109) • Bhavyaa Garg (PES1UG24CS116)

Chaithanya N (PES1UG24CS124) • Arabhi Bhat (PES1UG24CS927), Team Lead

UE24CS341A — Software Engineering, Semester V

Date: 30 September 2026

Table of Contents

Revision History

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions and Requirement Specification Method
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 Definitions, Acronyms and Abbreviations
- 1.6 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements

- 3.1 User Interfaces
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

4. Analysis Models

- 4.1 UML Use Case Diagram
- 4.2 Use Case Catalogue
- 4.3 Use Case Specifications
- 4.4 Data / Entity Outline

5. System Features (Functional Requirements)

- 5.1 Administrator Authentication & Access Control
- 5.2 Student Record Management (CRUD)
- 5.3 Search, Sort and Filter
- 5.4 Reports and Statistics
- 5.5 Data Persistence and Validation

6. Non-Functional Requirements

- 6.1 Performance Requirements
- 6.2 Reliability and Safety Requirements
- 6.3 Security Requirements
- 6.4 Software Quality Attributes
- 6.5 Business Rules

7. Other Requirements

Appendix A: Glossary

Appendix B: Field Layouts and Validation Rules

Revision History

Name	Date	Reason for Changes	Version
Team 17	Sep 2026	Initial draft for Requirements submission.	1.0
Team 17	30 Sep 2026	Restructured to IEEE 830 / 29148 layout; added requirement-specification method (1.2); rewrote all FRs and NFRs with measurable fit criteria; added UML use case diagram and use case specifications (Section 4); added Security Objectives and objective-mapped Security Requirements (6.3); added hardware interfaces, user documentation and full traceability matrix.	2.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of the Student Record Management System (SRMS), a console-based C/C++ application developed by Team 17 for the UE24CS341A Software Engineering Jackfruit Mini-Project. It defines the functional, non-functional and security requirements of the complete product (all modules listed in the Synopsis) and serves as the baseline against which the Design (Deliverable 3), Implementation and Test (Deliverable 4) work products will be traced and verified.

1.2 Document Conventions and Requirement Specification Method

Every requirement in Sections 5 and 6 is written using the following method so that it is **clear, unambiguous, concise, atomic, measurable and verifiable**, as recommended by IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018:

- **Unique identifier:** each requirement has a stable ID of the form REQ-<MODULE>-nn (functional), NFR-<CATEGORY>-nn (non-functional) or SEC-nn (security). IDs are never reused.
- **Imperative keyword:** “shall” denotes a mandatory requirement; “should” denotes a desirable but non-binding goal. “will” is used only for statements of fact. No other modal verbs are used in requirement statements.
- **Standard sentence form:** “The system shall <action> <object> <condition/constraint>.” One requirement per statement; compound “and/or” requirements are split.
- **Fit criterion:** every requirement carries a measurable acceptance condition (exact value, range, count, time limit or observable output) that a tester can check with a pass/fail result.
- **Banned vague terms:** words such as “fast”, “user-friendly”, “appropriate”, “valid” (without a stated rule), “etc.” and “some” are not used; where needed they are replaced by numeric limits defined in this document or Appendix B.
- **Priority:** High = required for the first release demo; Medium = required for final submission; Low = included if time permits in Sprint 6.
- **Verification method:** Test (T) = executed unit/integration/system test; Inspection (I) = code or document review; Demonstration (D) = observed operation; Analysis (A) = measurement or static-analysis report.
- **Traceability:** each requirement traces back to a use case (Section 4) and forward to a test case (Appendix C).

Typographic conventions: menu options and literal console text are shown in double quotes (e.g., “Add Record”); field names are shown in lower_snake_case (e.g., roll_no); bold text marks key terms.

1.3 Intended Audience and Reading Suggestions

This SRS is intended for: (a) the four development-team members, who use it to plan design, implementation and test cases; (b) course faculty and evaluators, who assess requirement quality and traceability; and (c) reviewers of later deliverables, who trace design and code back to the requirements defined here. Readers new to the product should read Sections 1 and 2 first, then the use case diagram in

Section 4. Developers and testers should focus on Sections 5 and 6 and Appendix B. Evaluators assessing security should read Section 6.3.

1.4 Product Scope

SRMS digitises student record keeping for a college department’s academic/examination office. It replaces paper registers and scattered spreadsheets with a single console-based C/C++ application backed by a persistent SQLite database. An authenticated Administrator can create, update, delete, list, search, sort, filter and report on student academic records; a Student/Viewer can look up one record read-only by roll number.

Objectives of the product: (1) eliminate duplicate and invalid records through validation at entry; (2) reduce the time to locate a record to under 1 second; (3) produce class lists, average-CGPA and topper reports without manual compilation; (4) ensure only an authenticated Administrator can change data. Out of scope: full university ERP (fees, hostel, admissions), graphical or web front-ends, integration with external portals, biometric attendance, cloud or multi-campus deployment, and email/SMS notifications.

1.5 Definitions, Acronyms and Abbreviations

Key terms are defined in Appendix A (Glossary). Frequently used acronyms: SRMS – Student Record Management System; SRS – Software Requirements Specification; CRUD – Create, Read, Update, Delete; CGPA – Cumulative Grade Point Average; UC – Use Case; FR – Functional Requirement; NFR – Non-Functional Requirement; CI – Continuous Integration; PBKDF2 – Password-Based Key Derivation Function 2.

1.6 References

- IEEE Std 830-1998, “IEEE Recommended Practice for Software Requirements Specifications.”
- ISO/IEC/IEEE 29148:2018, “Systems and software engineering — Life cycle processes — Requirements engineering.”
- PES University, Dept. of CSE — “Jackfruit Mini-Project: Software Engineering — Project Guidelines,” UE24CS341A, 2026.
- PES University, Dept. of CSE — “Software Engineering – Level 3 Mini-Project: Student Submission Guide & Complete Exemplar.”
- Team 17 — “SRMS Synopsis / Project Proposal,” Project ID SRMS-2026-17, Version 1.0.
- SQLite Documentation, sqlite.org — C/C++ Interface (sqlite3 library).
- OWASP Foundation — “Password Storage Cheat Sheet” and “SQL Injection Prevention Cheat Sheet.”
- OMG Unified Modeling Language (UML) Specification, Version 2.5.1.

2. Overall Description

2.1 Product Perspective

SRMS is a new, self-contained product; it is not a follow-on to an existing system and has no legacy system to integrate with. It runs as a single process on one machine: the user interacts through a text console, the application logic (C/C++) processes the request, and all data is stored in a single SQLite database file on local disk. There is no network or multi-tier component in this version. Internally the product is divided into five modules that match the team's ownership split: Authentication, Record CRUD, Search/Sort/Filter, Reports, and a shared Persistence & Validation layer.

2.2 Product Functions

At a high level, SRMS provides the following ten functions (detailed in Section 5):

- Administrator login/logout with role-based access control and account lockout.
- Add a new student record (roll no., name, department, semester, contact, CGPA).
- View / list all student records.
- Search a student record by roll number or name.
- Update an existing student record.
- Delete a student record after confirmation.
- Sort and filter records by department, semester or CGPA.
- Generate reports (class list, average CGPA, topper list) and export them to a file.
- Persist all data between sessions in a SQLite database.
- Validate input and prevent duplicate roll numbers or invalid data entry.

2.3 User Classes and Characteristics

User class	Privileges	Characteristics	Importance
Administrator	Full access: create, read, update, delete records; list, sort, filter; generate and export reports.	Office staff trained on the system; moderate computer skills; uses the system daily during registration and exam season.	Primary (favoured) user
Student / Viewer	Read-only search of one record by exact roll number; no login required; sees only non-sensitive fields.	Non-technical, infrequent user (a few times per semester); may use a shared front-desk terminal.	Secondary user

2.4 Operating Environment

- **Operating system:** Ubuntu 22.04 LTS or later (primary development and evaluation platform); Windows 10/11 with MinGW-w64 as a secondary target.
- **Compiler/toolchain:** gcc/g++ 11 or later, or clang 14 or later, with C++17 support; GNU Make or CMake 3.16+.

- **Libraries:** SQLite 3.31+ (embedded, via the sqlite3 C API); OpenSSL libcrypto 1.1.1+ (password hashing); GoogleTest 1.12+ (tests only).
- **Hardware (minimum):** x86-64 CPU, 2 GB RAM, 100 MB free disk, 80×24 character terminal.
- No web browser, network connection, database server or GUI toolkit is required.

2.5 Design and Implementation Constraints

- The implementation language shall be C/C++ (C++17), as assigned to Team 17.
- The user interface shall be a text-based console menu; no GUI is permitted in this version.
- Persistent storage shall use SQLite accessed only through the sqlite3 C API with prepared statements.
- Development shall follow Agile (six sprints) with backlog and sprint progress visible on GitHub Projects.
- Source control shall be Git/GitHub; every feature shall be merged via a reviewed pull request.
- A GitHub Actions CI pipeline shall build the project and run all unit tests on every push.
- Static analysis (cppcheck) shall be run before each sprint review.
- Only one Administrator session may be active against the database at a time (see 2.7).

2.6 User Documentation

- README.md in the repository: build, run and test instructions, and default first-login procedure.
- In-application help: a “Help” option on every menu listing the valid options and input formats for that screen.
- Administrator quick-reference guide (PDF, max. 4 pages) delivered with the final submission.

2.7 Assumptions and Dependencies

- The sqlite3 and OpenSSL development libraries and headers are available on the build/evaluation machine.
- Only one Administrator session is active at a time; concurrent multi-admin writes are out of scope for v1.0.
- The system is used by a single academic department, holding at most 10,000 student records.
- Roll numbers are unique within the department and are used as the primary key.
- The host machine’s system clock is correct; it is used for timestamps and lockout timing.
- The project depends on GitHub (repository, Projects board, Actions) being available during development.

3. External Interface Requirements

3.1 User Interfaces

SRMS presents numbered text menus on the console. The Main Menu offers: 1. Administrator Login, 2. Search Record (Viewer), 3. Help, 0. Exit. After login, the Administrator Menu offers: 1. Add Record, 2.

Update Record, 3. Delete Record, 4. View All Records, 5. Search Record, 6. Sort / Filter Records, 7. Reports, 8. Help, 9. Logout.

ID	User-interface requirement	Fit criterion	Verif.
UI-01	Every screen shall display a title line naming the current menu or operation.	Title visible on 100% of screens in the walkthrough checklist.	I
UI-02	The system shall accept a menu choice as a single number and re-display the menu when an out-of-range or non-numeric choice is entered.	Inputs “abc”, “-1”, “99” each re-display the menu without crashing.	T
UI-03	All error messages shall use the format “ERROR: <field>: <rule violated>.”	All error strings in source match the format (grep review).	I
UI-04	Tabular output shall fit within an 80-character-wide terminal, truncating names longer than 25 characters with “...”.	No output line exceeds 80 characters in system test.	T
UI-05	The system shall return to the previous menu after each operation completes or is cancelled.	Observed for all 13 use cases in demonstration.	D

3.2 Hardware Interfaces

SRMS has no direct hardware interfaces. It uses only standard keyboard input and terminal text output provided by the operating system, and reads/writes files on the local file system.

3.3 Software Interfaces

Component	Name / Version	Purpose / Data exchanged
Database engine	SQLite 3.31+ (embedded, sqlite3 C API)	Stores the students, admins and audit_log tables; accessed only through prepared statements.
Cryptography library	OpenSSL libcrypto 1.1.1+	PBKDF2-HMAC-SHA256 password hashing and random salt generation (RAND_bytes).
Compiler	gcc/g++ 11+ or clang 14+ (C++17)	Builds the application and tests.
Build system	GNU Make / CMake 3.16+	Compiles and links the application and test binaries.
Test framework	GoogleTest 1.12+	Unit and integration tests for all modules.
Static analysis	cppcheck 2.x	Detects defects and unsafe code before sprint reviews.
File system	POSIX / Windows file API	Writes exported reports (.txt/.csv) and the security log file.

3.4 Communications Interfaces

Not applicable for this version. SRMS is an offline, single-machine application with no network, email or web communication.

4. Analysis Models

This section models the required behavior of SRMS with a UML use case diagram and textual use case specifications. Class, sequence, activity and component diagrams will be produced in the Design deliverable.

4.1 UML Use Case Diagram

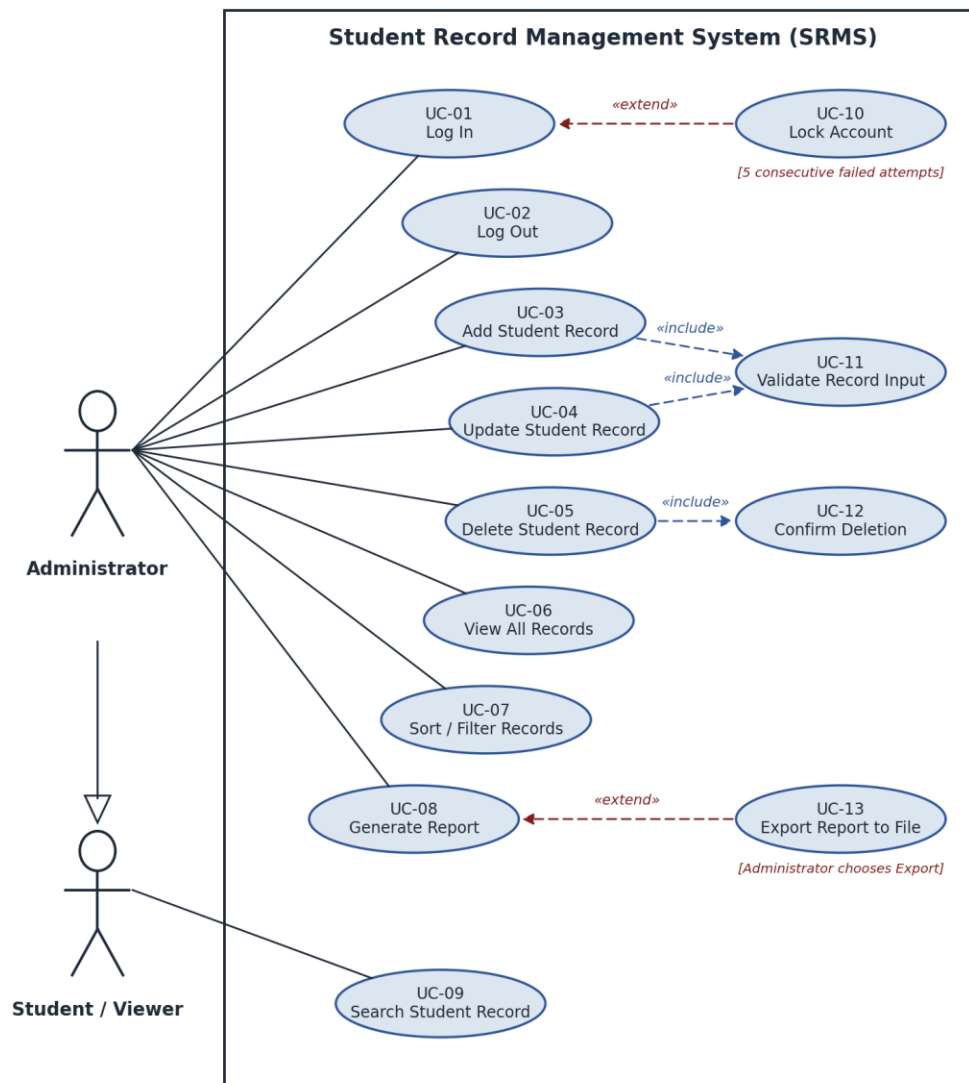


Figure 4.1 – UML Use Case Diagram of SRMS

Reading the diagram: solid lines are associations between an actor and the use cases it initiates. The hollow-triangle arrow from Administrator to Student/Viewer is a generalisation: the Administrator inherits every capability of the Viewer (UC-09 Search). Dashed «include» arrows point to behaviour that is always executed as part of the base use case (every Add and Update validates input; every Delete

requires confirmation). Dashed «extend» arrows point from optional behaviour to the base use case it extends, with the extension condition shown in brackets.

4.2 Use Case Catalogue

UC ID	Use case	Primary actor	Relationship	Related FRs
UC-01	Log In	Administrator	Extended by UC-10	REQ-AUTH-01..03, 06
UC-02	Log Out	Administrator	—	REQ-AUTH-07, 08
UC-03	Add Student Record	Administrator	Includes UC-11	REQ-CRUD-01, 02
UC-04	Update Student Record	Administrator	Includes UC-11	REQ-CRUD-03, 04, 07
UC-05	Delete Student Record	Administrator	Includes UC-12	REQ-CRUD-05, 07
UC-06	View All Records	Administrator	—	REQ-CRUD-06
UC-07	Sort / Filter Records	Administrator	—	REQ-SRCH-05..07
UC-08	Generate Report	Administrator	Extended by UC-13	REQ-RPT-01..03, 05
UC-09	Search Student Record	Student/Viewer (inherited by Administrator)	—	REQ-SRCH-01..04, REQ-AUTH-05
UC-10	Lock Account	System (on behalf of UC-01)	Extends UC-01	REQ-AUTH-04, SEC-03
UC-11	Validate Record Input	System	Included by UC-03, UC-04	REQ-DATA-02..05
UC-12	Confirm Deletion	System	Included by UC-05	REQ-CRUD-05
UC-13	Export Report to File	Administrator	Extends UC-08	REQ-RPT-04

4.3 Use Case Specifications

Detailed specifications are given for the three highest-risk use cases; the remaining use cases follow the stimulus/response sequences in Section 5.

UC-01 Log In

Primary actor	Administrator
Preconditions	Application is running and the Main Menu is displayed; the account is not locked.
Postconditions (success)	An Administrator session is active and the Administrator Menu is displayed; a LOGIN_SUCCESS event is logged.

Main success scenario	1. Administrator selects “Administrator Login”. 2. System prompts for username. 3. Administrator enters username. 4. System prompts for password and reads it without echoing characters. 5. System verifies the credentials against the stored salted hash. 6. System resets the failed-attempt counter, starts the session and displays the Administrator Menu.
Alternate flows	5a. Credentials invalid: system displays “ERROR: login: invalid username or password.”, increments the failed-attempt counter, logs LOGIN_FAILURE and returns to step 2. 5b. Fifth consecutive failure: UC-10 Lock Account is executed. 6a. First login with the default password: system forces a password change before showing the menu (SEC-06).
Related requirements	REQ-AUTH-01 to REQ-AUTH-04, REQ-AUTH-06; SEC-01, SEC-03, SEC-04, SEC-06

UC-03 Add Student Record

Primary actor	Administrator
Preconditions	Administrator session is active.
Postconditions (success)	A new row exists in the students table with created_at and updated_at set; a confirmation showing the roll number is displayed.
Main success scenario	1. Administrator selects “Add Record”. 2. System prompts for roll_no, name, department, semester, contact and cgpa in that order. 3. For each field, system executes UC-11 Validate Record Input. 4. System checks that roll_no does not already exist. 5. System inserts the record in a single transaction. 6. System displays “Record <roll_no> added successfully.”
Alternate flows	3a. Field fails validation: system shows the field-specific error and re-prompts for that field only (max. 3 attempts, then cancels). 4a. Duplicate roll_no: system displays “ERROR: roll_no: <value> already exists.” and cancels. 5a. Database write fails: system rolls back, displays “ERROR: database: record not saved.” and returns to the menu.
Related requirements	REQ-CRUD-01, REQ-CRUD-02, REQ-DATA-01 to REQ-DATA-05

UC-05 Delete Student Record

Primary actor	Administrator
Preconditions	Administrator session is active; at least one record exists.
Postconditions (success)	The record no longer exists in the database; a RECORD_DELETED event is logged.

Main success scenario	1. Administrator selects “Delete Record”. 2. System prompts for roll_no. 3. System displays the matching record. 4. System executes UC-12: prompts “Type YES to confirm deletion of <roll_no>:”. 5. Administrator types YES. 6. System deletes the record and displays “Record <roll_no> deleted.”
Alternate flows	3a. roll_no not found: system displays “ERROR: roll_no: record not found.” and returns to the menu. 5a. Any input other than exactly “YES”: system displays “Deletion cancelled.” and makes no change.
Related requirements	REQ-CRUD-05, REQ-CRUD-07, NFR-SAFE-01, SEC-07

4.4 Data / Entity Outline

SRMS persists three entities in one SQLite database file (srms.db). The full ER diagram and normalised schema will be finalised in the Design deliverable.

- **Student** (table students): roll_no (PK), name, department, semester, contact, cgpa, created_at, updated_at — see Appendix B.
- **Administrator** (table admins): username (PK), password_hash, salt, role, failed_attempts, locked_until, must_change_password.
- **AuditEvent** (table audit_log): event_id (PK), timestamp, username, event_type, target_roll_no, outcome.

5. System Features (Functional Requirements)

Features are organised to match the product-ownership split declared in the Synopsis, so each member's functional requirements map directly to their owned module. Each owner writes the unit tests for their own module; the CI pipeline (set up once by Arabhi Bhat) runs all tests on every push. All field formats referenced below are defined in Appendix B.

5.1 Administrator Authentication & Access Control

Owner: B Varun | Use cases: UC-01, UC-02, UC-09, UC-10

5.1.1 Description and Priority

Provides login and logout for the Administrator, enforces role-based access so that only an authenticated Administrator can modify data, and allows a Viewer to search without logging in. Priority: **High**.

5.1.2 Stimulus/Response Sequences

- User selects “Administrator Login” → system prompts for username and (non-echoed) password → on valid credentials, system starts an Administrator session and shows the Administrator Menu.
- User selects “Search Record” from the Main Menu without logging in → system grants Viewer access to UC-09 only.
- Administrator selects “Logout”, or is idle for 10 minutes → system ends the session and shows the Main Menu.

5.1.3 Functional Requirements

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-AUTH-01	The system shall grant Administrator privileges only after the user enters a username and password that match a stored Administrator account.	Login with correct credentials opens the Administrator Menu; login with any other pair does not (10/10 test pairs).	High	T
REQ-AUTH-02	The system shall display the single message “ERROR: login: invalid username or password.” for every failed login, regardless of which credential was wrong.	Wrong-username and wrong-password attempts produce byte-identical messages.	High	T
REQ-AUTH-03	The system shall not echo password characters to the console while they are typed.	No password characters (or asterisks revealing length) appear on screen during login.	High	D
REQ-AUTH-04	The system shall lock an Administrator account for 300 seconds after 5 consecutive failed login attempts.	6th attempt within 300 s is refused even with the correct password; correct login succeeds after 300 s.	High	T

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-AUTH-05	The system shall allow an unauthenticated Viewer to execute only the Search Record function (UC-09).	Main Menu without login offers no Add, Update, Delete, List, Sort/Filter or Report option.	High	T
REQ-AUTH-06	The system shall reject every Add, Update, Delete, List, Sort/Filter and Report request that is not made within an active Administrator session.	Direct calls to each of the 6 service functions without a session return an ACCESS_DENIED status in unit tests.	High	T
REQ-AUTH-07	The system shall end the Administrator session and return to the Main Menu when the Administrator selects “Logout”.	After logout, the next screen is the Main Menu and Admin options are unavailable.	Medium	T
REQ-AUTH-08	The system shall end an Administrator session automatically after 10 minutes without keyboard input.	Session ends between 600 s and 610 s of inactivity (test with configurable timer).	Medium	T

5.2 Student Record Management (CRUD)

Owner: Bhavyaa Garg | Use cases: UC-03, UC-04, UC-05, UC-06, UC-12

5.2.1 Description and Priority

Allows the Administrator to create, list, update and delete student academic records, the core data managed by SRMS. Priority: **High**.

5.2.2 Stimulus/Response Sequences

- Administrator selects “Add Record” → system prompts for each field → on valid, unique input, the record is saved and a confirmation with the roll number is shown.
- Administrator selects “Update Record” and enters a roll number → system shows the current values, accepts new values (Enter keeps the old value) and saves the changes.
- Administrator selects “Delete Record” and enters a roll number → system shows the record and asks the user to type YES → on YES, the record is deleted.
- Administrator selects “View All Records” → system lists records in pages of 20, ordered by roll number.

5.2.3 Functional Requirements

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-CRUD-01	The system shall create a new student record from the fields roll_no, name, department, semester, contact (optional) and cgpa entered by the Administrator.	After Add, the record is retrievable by UC-09 with all 6 field values identical to those entered.	High	T

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-CRUD-02	The system shall reject a new record whose roll_no already exists and display “ERROR: roll_no: <value> already exists.”	Adding an existing roll_no leaves the record count unchanged and shows the message.	High	T
REQ-CRUD-03	The system shall allow the Administrator to change any field except roll_no of an existing record identified by roll_no.	Each of the 5 editable fields can be changed and the change is visible after restart.	High	T
REQ-CRUD-04	The system shall keep the existing value of a field when the Administrator presses Enter without typing a new value during Update.	Pressing Enter on all prompts leaves the record unchanged except updated_at.	Medium	T
REQ-CRUD-05	The system shall delete a record only after the Administrator types exactly “YES” (case-sensitive) at the confirmation prompt.	Inputs “yes”, “Y”, blank and “NO” cancel deletion; “YES” deletes it.	High	T
REQ-CRUD-06	The system shall list all stored records ordered by roll_no ascending, 20 records per page, showing roll_no, name, department, semester and cgpa.	With 45 records, 3 pages are shown (20, 20, 5) in ascending roll_no order.	Medium	T
REQ-CRUD-07	The system shall display “ERROR: roll_no: record not found.” when an Update or Delete targets a roll_no that does not exist, and make no change.	Record count and contents are identical before and after the attempt.	Medium	T
REQ-CRUD-08	The system shall set created_at when a record is added and update updated_at on every successful update, using the format YYYY-MM-DD HH:MM:SS.	Timestamps match the system clock within 2 seconds.	Low	T

5.3 Search, Sort and Filter

Owner: Chaithanya N | Use cases: UC-07, UC-09

5.3.1 Description and Priority

Lets users locate specific records quickly by roll number or name, and lets the Administrator order or narrow the full record set by department, semester or CGPA. Priority: **High**.

5.3.2 Stimulus/Response Sequences

- User selects “Search Record” and enters a roll number (Viewer) or a roll number or name (Administrator) → system shows matching record(s) or a no-match message.
- Administrator selects “Sort”, a field (department, semester, cgpa) and an order (A = ascending, D = descending) → system shows all records in that order.

- Administrator selects “Filter” and enters a department, a semester, or both → system shows only the matching records.

5.3.3 Functional Requirements

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-SRCH-01	The system shall return the record whose roll_no exactly matches the entered value, ignoring letter case.	Searching “pes1ug24cs116” returns record PES1UG24CS116.	High	T
REQ-SRCH-02	The system shall, for an Administrator, return all records whose name contains the entered text (minimum 2 characters), ignoring letter case, ordered by roll_no.	“var” returns every name containing var/Var/VAR; a 1-character query is rejected.	Medium	T
REQ-SRCH-03	The system shall display “No matching record found.” when a search returns zero records.	Message shown for a non-existent roll_no and a non-matching name.	Medium	T
REQ-SRCH-04	The system shall display only roll_no, name, department, semester and cgpa to a Viewer; contact and timestamps shall not be shown.	Viewer search output contains no contact value in 5/5 test records.	High	T
REQ-SRCH-05	The system shall sort the full record list by department, semester or cgpa in ascending or descending order, as selected.	For each of the 6 field/order combinations the output is correctly ordered.	High	T
REQ-SRCH-06	The system shall break sort ties by roll_no ascending.	Two records with equal cgpa appear in ascending roll_no order.	Low	T
REQ-SRCH-07	The system shall filter the record list by department, by semester, or by both combined, showing only records matching every given criterion.	Filter CSE + semester 5 returns exactly the matching subset of the test dataset.	High	T

5.4 Reports and Statistics

Owner: Arabhi Bhat (Team Lead) | Use cases: UC-08, UC-13

5.4.1 Description and Priority

Produces summary academic reports from the stored records and exports them to a file, replacing manual report compilation. Priority: **Medium**.

5.4.2 Stimulus/Response Sequences

- Administrator selects “Reports”, a report type (Class List / Average CGPA / Topper List) and a department and semester → system computes and displays the report.
- Administrator selects “Export” after viewing a report and chooses TXT or CSV → system writes the file and shows its full path.

5.4.3 Functional Requirements

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-RPT-01	The system shall generate a Class List of all students in a selected department and semester, ordered by roll_no, showing the fields in Appendix B.2.	Output rows equal the count returned by the equivalent SQL query on the test dataset.	Medium	T
REQ-RPT-02	The system shall compute the average cgpa for a selected department and semester, rounded to 2 decimal places.	Result equals the hand-computed mean (± 0.005) for 3 test datasets.	Medium	T
REQ-RPT-03	The system shall list every student having the highest cgpa in a selected department and semester, including all students tied for highest.	Dataset with a 2-way tie lists both students.	Medium	T
REQ-RPT-04	The system shall export the displayed report to <code>./reports/<type>_<YYYYMMDD_HHMMSS>.txt</code> or <code>.csv</code> , as chosen, and display the file path.	File exists at the displayed path and its data rows match the on-screen report.	Low	T
REQ-RPT-05	The system shall display “No records available for this report.” and create no file when the selected department and semester contain zero records.	Empty selection shows the message; <code>./reports/</code> gains no new file.	Low	T

5.5 Data Persistence and Validation

Owner: Shared — SQL schema & queries: Chaithanya N; input validation helpers: Bhavyaa Garg | Use case: UC-11

5.5.1 Description and Priority

Ensures all data survives across program runs, and that data is validated at the point of entry to prevent duplicate or invalid records. Priority: **High**.

5.5.2 Stimulus/Response Sequences

- Administrator adds, updates or deletes a record → system commits the change to `srms.db` before displaying the success message.
- Application restarts → system opens the existing `srms.db` and all previously committed records are available unchanged.
- User enters a value that breaks a rule in Appendix B → system rejects it immediately and re-prompts for that field with a specific error.

5.5.3 Functional Requirements

ID	Requirement	Fit Criterion (measurable acceptance)	Priority	Verif.
REQ-DATA-01	The system shall commit every Add, Update and Delete to the SQLite file srms.db before reporting success to the user.	After a successful operation and a forced kill (kill -9), the change is present on restart.	High	T
REQ-DATA-02	The system shall reject a roll_no that does not match the pattern <code>^[A-Z0-9]{5,20}\$</code> after conversion to upper case.	“PES1UG24CS116” accepted; “AB1”, “PES-116” and blank rejected.	High	T
REQ-DATA-03	The system shall reject a name that is empty, longer than 80 characters, or contains characters other than letters, spaces, period, hyphen and apostrophe.	“A”, “Anne-Marie O’Neil” and an 80-char name accepted; blank, 81 chars and “R2D2” rejected.	High	T
REQ-DATA-04	The system shall reject a semester outside the integers 1 to 8 and a cgpa outside 0.00 to 10.00 or with more than 2 decimal places.	0, 9, 2.5 (semester) and -0.01, 10.01, 8.123 (cgpa) rejected; 1, 8, 0.00, 10.00 accepted.	High	T
REQ-DATA-05	The system shall accept a department only if it is one of the codes stored in the departments list (default: CSE, ECE, EEE, ME, CV, BT, AIML).	“CSE” accepted; “XYZ” rejected with the list of valid codes shown.	High	T
REQ-DATA-06	The system shall enforce roll_no uniqueness through a PRIMARY KEY constraint on the students table.	A direct duplicate INSERT via the sqlite3 CLI fails with a constraint error.	High	I
REQ-DATA-07	The system shall, on any database read or write error, display “ERROR: database: <operation> failed.”, roll back the transaction and return to the menu without terminating.	With srms.db set read-only, Add shows the message and the program keeps running.	Medium	T
REQ-DATA-08	The system shall create srms.db with the full schema automatically if the file does not exist at start-up.	Deleting srms.db and starting the program produces a working empty database.	Medium	T

6. Non-Functional Requirements

All performance figures are measured on the reference evaluation machine (4-core x86-64 CPU, 8 GB RAM, SSD, Ubuntu 22.04) with a test database of 10,000 synthetic records, timed from the moment Enter is pressed to the moment the first line of the result is printed, averaged over 20 runs.

6.1 Performance Requirements

ID	Requirement	Measurement / Fit criterion	Verif.
NFR-PERF-01	The system shall return the result of a search by roll_no within 0.5 seconds.	Mean ≤ 0.5 s, max ≤ 1 s over 20 runs.	Test (A)
NFR-PERF-02	The system shall return the result of a name search, sort or filter within 2 seconds.	Mean ≤ 2 s over 20 runs for each operation.	Test (A)
NFR-PERF-03	The system shall generate any report within 2 seconds.	Mean ≤ 2 s over 20 runs for each of the 3 report types.	Test (A)
NFR-PERF-04	The system shall complete an Add, Update or Delete commit within 1 second.	Mean ≤ 1 s over 20 runs.	Test (A)
NFR-PERF-05	The system shall start and display the Main Menu within 3 seconds.	Measured from process launch; mean ≤ 3 s.	Test (A)
NFR-PERF-06	The application shall use no more than 50 MB of resident memory.	Peak RSS ≤ 50 MB measured with /usr/bin/time -v.	Analysis

6.2 Reliability and Safety Requirements

ID	Requirement	Measurement / Fit criterion	Verif.
NFR-SAFE-01	The system shall require typed confirmation ("YES") before any irreversible deletion.	See REQ-CRUD-05; 0 deletions occur without YES.	Test
NFR-REL-01	The system shall execute each write inside a SQLite transaction so that an unexpected termination leaves the database in the state before or after the operation, never partially written.	20 forced kills during writes; PRAGMA integrity_check returns "ok" each time.	Test
NFR-REL-02	The system shall not terminate abnormally on any user input.	0 crashes over a 1,000-input randomised (fuzz) test of all menus.	Test
NFR-REL-03	The system shall run 500 consecutive mixed operations without failure or memory leak.	0 errors; Valgrind reports 0 bytes "definitely lost".	Test (A)

ID	Requirement	Measurement / Fit criterion	Verif.
NFR-REL-04	The system shall produce identical output for identical input and database state.	Two runs of the scripted system test produce byte-identical output (excluding timestamps).	Test

6.3 Security Requirements

This section defines the security objectives of SRMS (what must be protected and why) and the security requirements that realise them. Student records contain personal data (names, contact details, academic results); the objectives follow the CIA triad plus authentication and accountability.

6.3.1 Security Objectives

ID	Objective	Statement	Asset protected
SO-01	Confidentiality	Personal and academic data shall be disclosed only to authorised users: full records to an authenticated Administrator, and only non-sensitive fields of a single record to a Viewer.	Student records, contact details, admin credentials
SO-02	Integrity	Student records shall be created, modified or deleted only by an authenticated Administrator through validated operations, and shall never be altered by crafted input.	Student records, database file
SO-03	Authentication & Authorisation	Every privileged operation shall be traceable to a verified Administrator identity, and privileges shall be granted strictly by role.	Admin accounts, privileged functions
SO-04	Accountability	Security-relevant events shall be recorded so that any change or access attempt can be reviewed after the fact.	Audit trail
SO-05	Availability & Resilience	Authorised users shall be able to access correct data after crashes, invalid input or brute-force attempts.	Database file, running service

6.3.2 Security Requirements

ID	Security requirement	Obj.	Fit criterion	Validation
SEC-01	The system shall store Administrator passwords only as PBKDF2-HMAC-SHA256 hashes with a unique 16-byte random salt per account and at least 100,000 iterations.	SO-01, SO-03	admins table contains no plaintext password; salts differ for 2 accounts with identical passwords.	Inspection + Test

ID	Security requirement	Obj.	Fit criterion	Validation
SEC-02	The system shall execute every SQL statement that contains user-supplied data as a parameterised prepared statement (sqlite3_prepare_v2 + sqlite3_bind_*); string concatenation of user input into SQL is prohibited.	SO-02	Code review finds 0 concatenated queries; inputs ' OR '1'='1 and ; DROP TABLE students;-- are stored/searched literally and all tables remain intact.	Inspection + Security test
SEC-03	The system shall lock an account for 300 seconds after 5 consecutive failed login attempts and reset the counter after a successful login.	SO-03, SO-05	As REQ-AUTH-04; counter = 0 after success.	Security test
SEC-04	The system shall record each LOGIN_SUCCESS, LOGIN_FAILURE, ACCOUNT_LOCKED, RECORD_ADDED, RECORD_UPDATED, RECORD_DELETED and REPORT_EXPORTED event in audit_log with timestamp, username, event type, target roll_no and outcome.	SO-04	Scripted session of all 7 events yields exactly 7 matching audit_log rows.	System test
SEC-05	The system shall never write passwords, password hashes or salts to the console, log or export files.	SO-01	grep for test password and hash values in all output and log files returns 0 matches.	Security test
SEC-06	The system shall force the Administrator to change the default password at first login; the new password shall be 8–64 characters with at least one letter and one digit.	SO-03	Default password cannot reach the Admin Menu without change; “password” and “12345678” rejected.	Test
SEC-07	The system shall verify an active Administrator session inside every data-modifying service function, not only at the menu layer.	SO-02, SO-03	Unit tests calling each function without a session receive ACCESS_DENIED (6/6).	Unit test
SEC-08	The system shall create srms.db and the audit log with owner-only file permissions (0600) on Linux.	SO-01, SO-02	ls -l shows -rw----- for both files after first run.	Inspection
SEC-09	The system shall show a Viewer only roll_no, name, department, semester and cgpa for one record per search.	SO-01	As REQ-SRCH-04.	Test
SEC-10	The system shall reject any input longer than its field limit in Appendix B before it is copied into a program buffer.	SO-02, SO-05	300-character input to every prompt causes no crash and no truncated write; cppcheck reports 0 buffer-overflow warnings.	Test + Analysis

6.3.3 Threats Considered and Mitigations

Threat	Mitigating requirements
SQL injection through search, add or update fields	SEC-02, SEC-10, REQ-DATA-02..05

Threat	Mitigating requirements
Brute-force guessing of the Administrator password	SEC-03, SEC-06, REQ-AUTH-02
Disclosure of credentials if the database file is copied	SEC-01, SEC-05, SEC-08
Viewer at a shared terminal reading other students' personal data	SEC-09, REQ-AUTH-05, REQ-AUTH-08
Unauthorised or disputed record changes	SEC-04, SEC-07
Buffer overflow via over-long console input	SEC-10, NFR-REL-02

6.4 Software Quality Attributes

ID	Requirement	Measurement / Fit criterion	Verif.
NFR-USE-01	A first-time Administrator shall be able to add a record using the on-screen prompts alone within 5 menu interactions.	4 of 5 novice test users succeed in ≤ 5 interactions and ≤ 2 minutes, without help.	Demo.
NFR-USE-02	Every validation error message shall name the field and the rule violated.	100% of error strings follow UI-03.	Inspection
NFR-MNT-01	Business logic (validation, search, sort, reports, auth) shall be in modules separate from console I/O code.	No file in src/core/ includes <iostream> or calls printf/scanf.	Inspection
NFR-MNT-02	No function shall exceed a cyclomatic complexity of 10 or a length of 60 lines.	lizard/cppcheck report shows 0 violations.	Analysis
NFR-MNT-03	The code shall produce 0 errors and 0 warnings with -Wall -Wextra and 0 high-severity cppcheck findings.	CI build log and cppcheck report.	Analysis
NFR-TST-01	Unit tests shall achieve at least 70% line coverage of src/core/.	gcov/lcov report $\geq 70\%$.	Analysis
NFR-TST-02	The CI pipeline shall build and run all tests on every push and finish within 10 minutes.	GitHub Actions history shows a run per push, each ≤ 10 min.	Inspection
NFR-PORT-01	The code shall build and pass all tests on Ubuntu 22.04 (gcc) and Windows 11 (MinGW-w64) without source changes.	Both CI/manual builds pass.	Test

6.5 Business Rules

- BR-01: Only an authenticated Administrator may create, update or delete a student record.
- BR-02: Every student record must have a unique roll_no; roll_no cannot be changed after creation.
- BR-03: A record cannot be saved with a missing mandatory field or a value outside the rules in Appendix B.

- BR-04: Deleting a record requires the Administrator to type YES.
- BR-05: Reports are computed only from records present in the database at the time of generation.
- BR-06: A Viewer may see only non-sensitive fields of one record per search.

7. Other Requirements

- **Database:** the SQLite schema shall be version-controlled as schema.sql in the repository and applied automatically on first run (REQ-DATA-08).
- **Legal/institutional:** student data shall be treated as internal academic records; the only route by which data leaves the application is the Report Export function (REQ-RPT-04), and every export is logged (SEC-04).
- **Reuse:** the sqlite3, OpenSSL libcrypto and GoogleTest libraries are reused third-party components; all other logic is developed by the team.
- **Internationalisation:** out of scope for v1.0; all text is in English.

Appendix A: Glossary

Term	Definition
Administrator	The authenticated user role with full create/read/update/delete, sort/filter and reporting privileges.
Viewer	An unauthenticated user permitted to search one record by roll_no and see non-sensitive fields only.
Roll number (roll_no)	The unique identifier of a student record; primary key of the students table.
CGPA	Cumulative Grade Point Average, a number from 0.00 to 10.00 stored per record.
CRUD	Create, Read, Update, Delete — the four core data operations on a record.
Fit criterion	A measurable condition that determines whether a requirement has been met.
Prepared statement	A pre-compiled SQL statement whose parameters are bound separately, preventing SQL injection.
PBKDF2	Password-Based Key Derivation Function 2; a slow, salted hashing scheme for storing passwords.
Salt	Random bytes combined with a password before hashing so identical passwords yield different hashes.
Session	The period between a successful Administrator login and logout or inactivity timeout.
SQLite	An embedded, file-based SQL database engine accessed via the sqlite3 C API.

Appendix B: Field Layouts and Validation Rules

B.1 Student record

Field	Type	Mandatory	Validation rule	Example
roll_no	TEXT (5–20)	Y	Upper-cased; ^[A-Z0-9]{5,20}\$; unique	PES1UG24CS116
name	TEXT (1–80)	Y	Letters, space, . - ' only; leading/trailing spaces trimmed	Bhavyaa Garg
department	TEXT	Y	One of: CSE, ECE, EEE, ME, CV, BT, AIML	CSE
semester	INTEGER	Y	Integer 1–8	5
contact	TEXT (≤50)	N	Either 10 digits starting 6–9, or an e-mail matching ^[^\s]+@[^\s]+\.[^\s]+\$	9876543210
cgpa	REAL	Y	0.00–10.00, at most 2 decimal places	8.41
created_at	TEXT	System	YYYY-MM-DD HH:MM:SS, set on insert	2026-09-30 10:15:00
updated_at	TEXT	System	YYYY-MM-DD HH:MM:SS, set on every update	2026-09-30 11:02:44

B.2 Report fields

Class List report	Average CGPA report	Topper List report
Roll number	Department	Department
Name	Semester	Semester
Department	Number of students	Highest CGPA
Semester	Average CGPA (2 d.p.)	Roll number(s) of topper(s)
CGPA	Generated-at timestamp	Name(s) of topper(s)

Appendix C: Requirement Traceability Matrix

Design, code and test-case references will be completed in Deliverables 3 and 4. Test-case IDs below are reserved now so that every requirement has a planned verification.

Requirement IDs	Description	Use case(s)	Owner	Planned test cases
REQ-AUTH-01..08	Authentication & access control	UC-01, 02, 09, 10	B Varun	TC-AUTH-01..08
REQ-CRUD-01..08	Student record CRUD	UC-03..06, 12	Bhavyaa Garg	TC-CRUD-01..08
REQ-SRCH-01..07	Search, sort and filter	UC-07, 09	Chaithanya N	TC-SRCH-01..07
REQ-RPT-01..05	Reports and statistics	UC-08, 13	Arabhi Bhat	TC-RPT-01..05
REQ-DATA-01..08	Persistence and validation	UC-03, 04, 11	Chaithanya N / Bhavyaa Garg	TC-DATA-01..08
UI-01..05	User interface	All	All	TC-UI-01..05
NFR-PERF-01..06	Performance	UC-03..09	All	TC-PERF-01..06
NFR-SAFE-01, NFR-REL-01..04	Safety and reliability	UC-03..05	All	TC-REL-01..05
SEC-01..10	Security (SO-01..05)	UC-01, 03..05, 09, 10	B Varun (lead) / All	TC-SEC-01..10
NFR-USE, MNT, TST, PORT	Quality attributes	—	All	Review / CI reports

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Architecture and Design Specification

Project: Student Record Management System (SRMS)

Version: 1.0

Authors: **Team 17 (Project ID SRMS-2026-17)**

B Varun (PES1UG24CS109) • Bhavyaa Garg (PES1UG24CS116)

Chaithanya N (PES1UG24CS124) • Arabhi Bhat (PES1UG24CS927), Team Lead

Date: 1 October 2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	1 Oct 2026	Team 17	Initial draft: architecture, component diagram, 4 sequence diagrams, API design, security architecture.

Approvals

Role	Name	Signature/Date	
Team Lead	Arabhi Bhat		
Course Coordinator			

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Student Record Management System (SRMS) v1.0. It describes the component structure, the chosen architecture pattern and its rationale, the dynamic behaviour of the system's four core flows (as UML sequence diagrams), the module interface (API) design, error-handling strategy, and the security architecture that realises the objectives and requirements defined in the SRS (v2.0). It is the Design-phase deliverable that developers use to implement each owned module consistently.

1.2 Scope

Covers the internal software architecture of SRMS v1.0: the layered component structure (Presentation → Controller → Service → Data Access → Data); sequence diagrams for Administrator Login, Add Student Record, Search Student Record and Generate/Export Report; the API (function-interface) design for the Auth, Record, Search and Report services; and the security architecture addressing the threats identified in SRS Section 6.3.3. Excludes hardware design (SRMS has no custom hardware interfaces) and any future GUI/web front-end, which is explicitly out of scope for v1.0 per the Synopsis.

1.3 Audience

Primary audience: the four Team 17 developers (B Varun, Bhavyaa Garg, Chaithanya N, Arabhi Bhat), each implementing the module they own. Secondary audience: course faculty and evaluators assessing architectural soundness, modularity and requirements traceability; and any future maintainer extending SRMS (e.g., adding a GUI layer).

1.4 Definitions

Acronyms and terms used here follow SRS Appendix A. Additional architecture-specific terms:

- DAO (Data Access Object): the module that issues parameterized SQL statements against SQLite on behalf of exactly one entity (StudentDAO, AdminDAO).
- Service: a module containing business logic and validation for one owned feature, with no direct console I/O (NFR-MNT-01).
- Controller: the thin layer that reads console menu choices and dispatches to the corresponding Service.
- STRIDE: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — the threat-modelling categories used in Section 3.9.

2. Document Overview

2.1 How to use this document

Each developer should read Section 3 (Architecture) before writing any code for their module, to confirm which layer their functions belong in and which other components they call. Before implementing a specific flow, read the matching sequence diagram and API table in Section 4.2–4.3. Testers should read Section 3.9 and 4.4 when designing security and negative test cases for the Test Plan. Evaluators can use Section 3.8 (Traceability to Requirements) to check that every SRS requirement has an architectural home.

2.2 Related Documents

- Software Requirements Specification (SRS), SRMS, Version 2.0 — source of all REQ/NFR/SEC/UC IDs referenced throughout this document.
- Software Test Plan (STP), SRMS, Version 1.0 — consumes this document's component/interface design for test-case design and the RTM.
- Team 17 — SRMS Synopsis / Project Proposal, Version 1.0.

3. Architecture

3.1 Goals & Constraints

Goals: a modular, maintainable, independently-testable layered design in which each of the four owned features (Auth, CRUD, Search, Reports) can be implemented, unit-tested and reviewed in isolation; secure-by-default behaviour realising security objectives SO-01 to SO-05; response times within the NFR-PERF-01..06 targets even at the 10,000-record reference scale.

Constraints: implementation language restricted to C/C++ (C++17); console-only UI, no GUI; SQLite as the only persistent store, accessed solely through the sqlite3 C API with prepared statements; only one Administrator session active at a time (SRS Section 2.7); the codebase must build and pass all tests unchanged on both Ubuntu 22.04 (gcc) and Windows 11 (MinGW-w64), per NFR-PORT-01.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Administrator (end user)	Accurate, up-to-date records; fast search/sort/filter; simple, forgiving console menus; confidence that deletions cannot happen by accident.
Course faculty / evaluators	Clean, modular design matching individual ownership; clear requirements-to-architecture-to-code traceability; demonstrable, executed security validation.
Development team (Team 17)	Well-defined interfaces between the four owned modules so integration doesn't block any one person; straightforward unit testing without a running console.
Future maintainer	Ability to add a GUI or networked front-end later without rewriting the Service/DAO layers (per Synopsis 'Future Enhancements').

3.3 Component (UML) Diagram

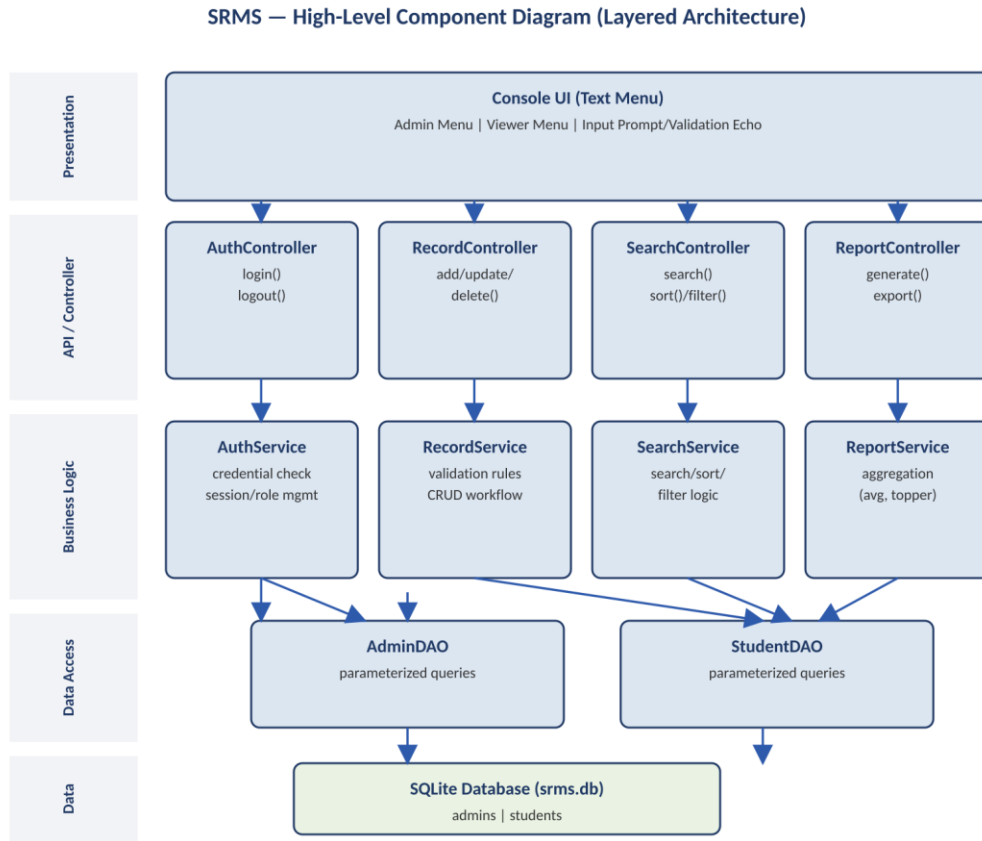


Figure 3.1 — SRMS High-Level Component Diagram (Layered Architecture)

Two cross-cutting Business Logic components are shared by every Service and omitted above for clarity: Validator (field/record validation) and AuditLogger (writes to `audit_log`). Both are detailed in Section 3.4.

3.4 Component Descriptions

Component	Responsibility	Owner
Console UI	Renders numbered text menus (Main / Administrator), reads input, echoes validation errors per UI-01..05.	B Varun
AuthController / AuthService	Login/logout, credential verification, session and role management, account lockout (REQ-AUTH, SEC-01, 03, 04, 06, 07, 08).	B Varun
RecordController / RecordService	Add/Update/Delete/List workflow and field validation orchestration (REQ-CRUD, REQ-DATA).	Bhavyaa Garg
Validator	Per-field and whole-record validation rules from SRS Appendix B; shared by every Service before a write (REQ-DATA-02..05).	Bhavyaa Garg
AuditLogger	Writes RECORD_ADDED / RECORD_UPDATED / RECORD_DELETED / login events to the <code>audit_log</code> table (SEC-04); used by all Services.	Shared (defined with CRUD)

Component	Responsibility	Owner
SearchController / SearchService	Search by roll no./name, sort, filter, and Viewer field-visibility enforcement (REQ-SRCH, SEC-09).	Chaithanya N
ReportController / ReportService	Class list / average CGPA / topper computation and file export (REQ-RPT).	Arabhi Bhat
AdminDAO, StudentDAO	Only components permitted to touch SQL; issue exclusively parameterized statements (SEC-02); schema owner.	Chaithanya N (schema) / shared
SQLite Database (srms.db)	Persistent storage for students, admins and audit_log tables (SRS Section 4.4); file permissions 0600 (SEC-08).	Shared

3.5 Chosen Architecture Pattern and Rationale

Pattern: classic layered architecture (Presentation → API/Controller → Business Logic → Data Access → Data), with strictly one-directional, adjacent-layer-only calls.

Rationale: the layering maps one-to-one onto the team's product-ownership split (Synopsis Section 1), so each member can implement, unit-test and demo their Controller+Service pair independently. Keeping console I/O (Presentation) separate from business logic (Service) satisfies NFR-MNT-01 and makes Services testable with GoogleTest without any console interaction. Centralising all SQL in two DAOs makes SEC-02 (parameterized queries only) easy to enforce and review in one place. A microservices or event-driven style was considered and rejected: SRMS is a single-process console tool for one department (complexity rated 'Easy' by the course), so the operational overhead of multiple processes or a message bus would add risk and complexity with no corresponding benefit.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology	Notes
Language / compiler	C++17, gcc/g++ 11+ or clang 14+	Per SRS Section 2.4
Build system	GNU Make or CMake 3.16+	Single build graph for app + tests
Persistence	SQLite 3.31+ via sqlite3 C API	Single file srms.db; 3 tables (students, admins, audit_log)
Cryptography	OpenSSL libcrypto 1.1.1+	PBKDF2-HMAC-SHA256 password hashing (SEC-01)
Testing	GoogleTest 1.12+	Unit tests per module, ≥70% line coverage (NFR-TST-01)
Static analysis	cppcheck 2.x	Run before every sprint review (NFR-MNT-03)
CI	GitHub Actions	Build + run all tests on every push (NFR-TST-02)

3.7 Risks & Mitigations

Risk	Mitigation
SQL injection through search/add/update fields	All SQL is parameterized (SEC-02); input validated against Appendix B rules before reaching the DAO (REQ-DATA-02..05).
Credential or password-hash leakage	PBKDF2 + per-account salt (SEC-01); never written to logs/console (SEC-05); srms.db and audit log created with 0600 permissions (SEC-08).
Single-admin-session assumption violated in practice	Every write wrapped in a SQLite transaction (NFR-REL-01); limitation documented as an explicit assumption (SRS Section 2.7), not silently handled.
Cross-platform build breakage (Ubuntu vs. Windows/MinGW)	CI matrix builds both targets; no platform-specific APIs used outside a small, isolated platform-adaptor file (NFR-PORT-01).
Schema drift between developers' machines	schema.sql is version-controlled and applied automatically on first run if srms.db does not exist (REQ-DATA-08).

3.8 Traceability to Requirements

Architecture Component	Requirements Realised
AuthController / AuthService	REQ-AUTH-01..08; SEC-01, 03, 04, 06, 07, 08; UC-01, 02, 10
RecordController / RecordService / Validator / AuditLogger	REQ-CRUD-01..08; REQ-DATA-02..05; SEC-04, 07; UC-03, 04, 05, 06, 11, 12
SearchController / SearchService	REQ-SRCH-01..07; SEC-09; UC-07, 09
ReportController / ReportService	REQ-RPT-01..05; UC-08, 13
AdminDAO / StudentDAO / SQLite DB	REQ-DATA-01..08; SEC-02, 08, 10
Console UI (all Controllers)	UI-01..05; NFR-USE-01, 02

3.9 Security Architecture

Threats are analysed using STRIDE and mapped to the architecture components responsible for mitigating each (consistent with the threat table in SRS Section 6.3.3):

STRIDE Category	Threat (SRMS-specific)	Mitigating Component(s) / Requirement
Spoofing	An attacker impersonates the Administrator	AuthService salted-hash verification (SEC-01); forced change of default password (SEC-06)
Tampering	Student records altered outside normal workflow	RecordService validation + transactional DAO writes (NFR-REL-01); session check

STRIDE Category	Threat (SRMS-specific)	Mitigating Component(s) / Requirement
		inside every data-modifying function, not just the menu (SEC-07)
Repudiation	An Administrator denies having made a change	AuditEvent logging of every state-changing action in AdminDAO/StudentDAO write path (SEC-04)
Information Disclosure	SQL injection, or a Viewer seeing sensitive fields	Parameterized statements in all DAOs only (SEC-02); SearchService field-filtering for Viewer role (SEC-09, REQ-SRCH-04)
Denial of Service	Brute-force login attempts; oversized input crashing the console	AuthService account lockout after 5 failures (SEC-03); input length checked before any buffer copy (SEC-10)
Elevation of Privilege	A Viewer session invokes an Administrator-only function directly	Role check inside every Service function (SEC-07), not solely at the Controller/menu layer, per REQ-AUTH-06

4. Design

4.1 Design Overview

SRMS is designed as four vertical, independently-ownable feature slices (Authentication, Record CRUD, Search/Sort/Filter, Reports) sitting on top of a shared Persistence & Validation layer, each slice following the same Controller → Service → DAO shape described in Section 3. This section details the dynamic behaviour of the four core user-facing flows as UML sequence diagrams, followed by the module (API) interface design, the error-handling strategy, and the console UX design.

4.2 UML Sequence Diagrams

Four sequence diagrams are provided — one per owned module — covering the highest-value flow in each (exceeding the minimum of two required).

4.2.1 Administrator Login (Owner: B Varun)

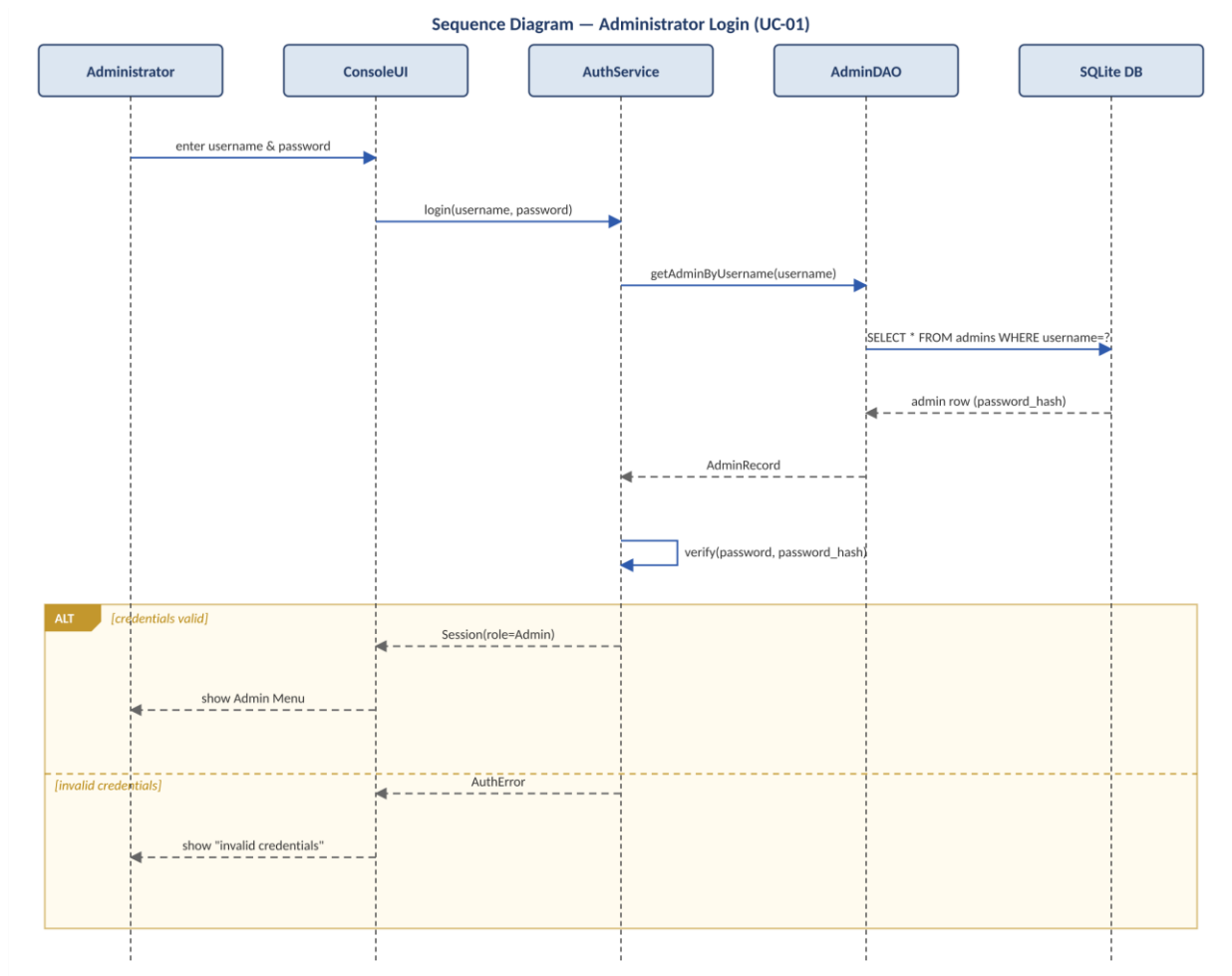


Figure 4.1 — Sequence Diagram: Administrator Login (UC-01; REQ-AUTH-01..04)

4.2.2 Add Student Record (Owner: Bhavyaa Garg)

Sequence Diagram 3 - Add Student Record (UC-03)

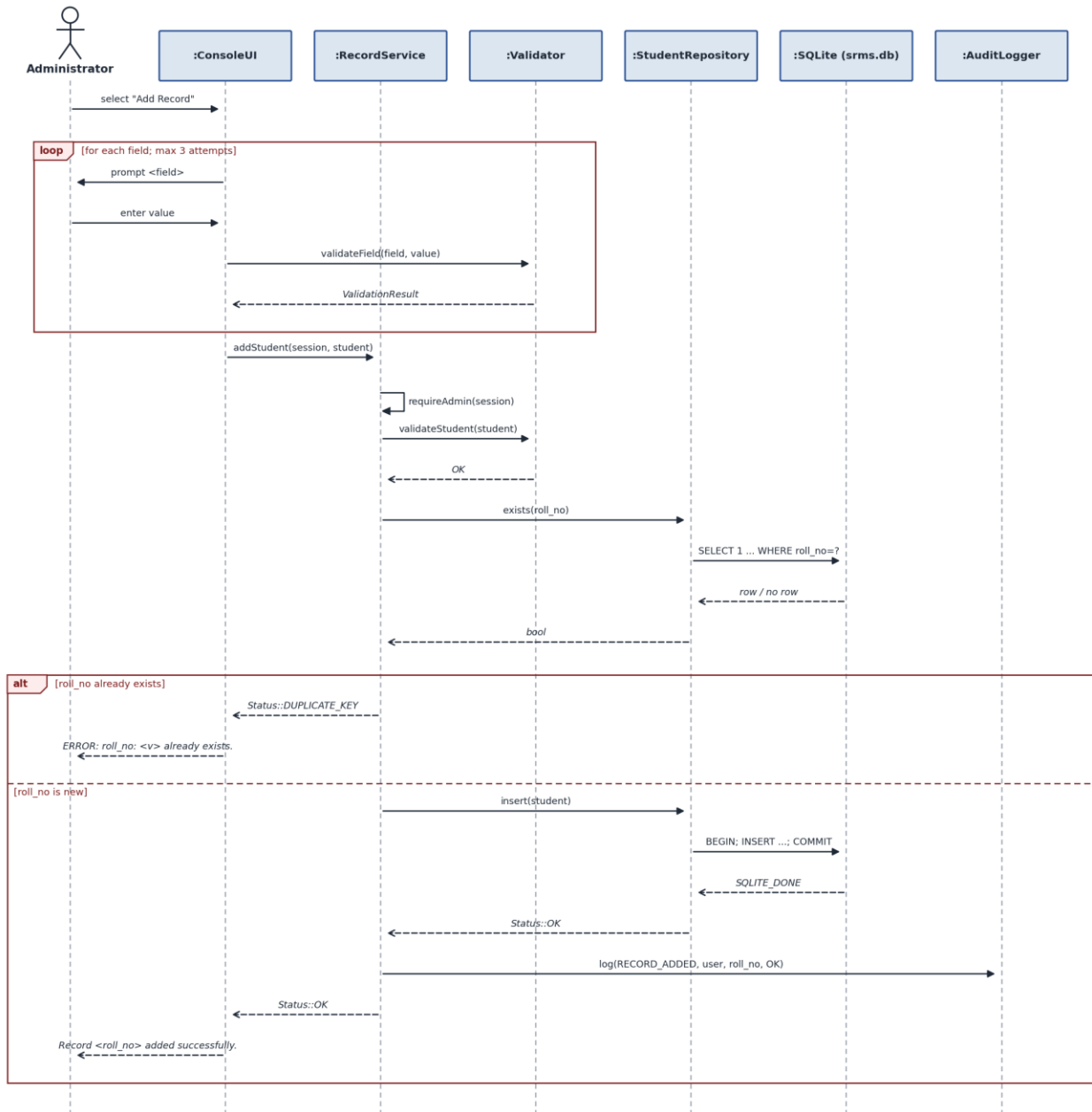


Figure 4.2 — Sequence Diagram: Add Student Record (UC-03, includes UC-11; REQ-CRUD-01, 02, 08; REQ-DATA-01..06; SEC-02, 04, 07)

ConsoleUI validates each field through Validator::validateField() as it is entered (max 3 attempts per field, else CANCELLED). RecordService::addStudent() re-checks the session (requireAdmin, SEC-07) and re-validates the whole record server-side as defence in depth, then StudentRepository::exists() runs a parameterized SELECT before StudentRepository::insert() commits the row as a single transaction and AuditLogger records RECORD_ADDED (SEC-04).

4.2.3 Update Student Record (Owner: Bhavyaa Garg)

Sequence Diagram 4 - Update Student Record (UC-04)

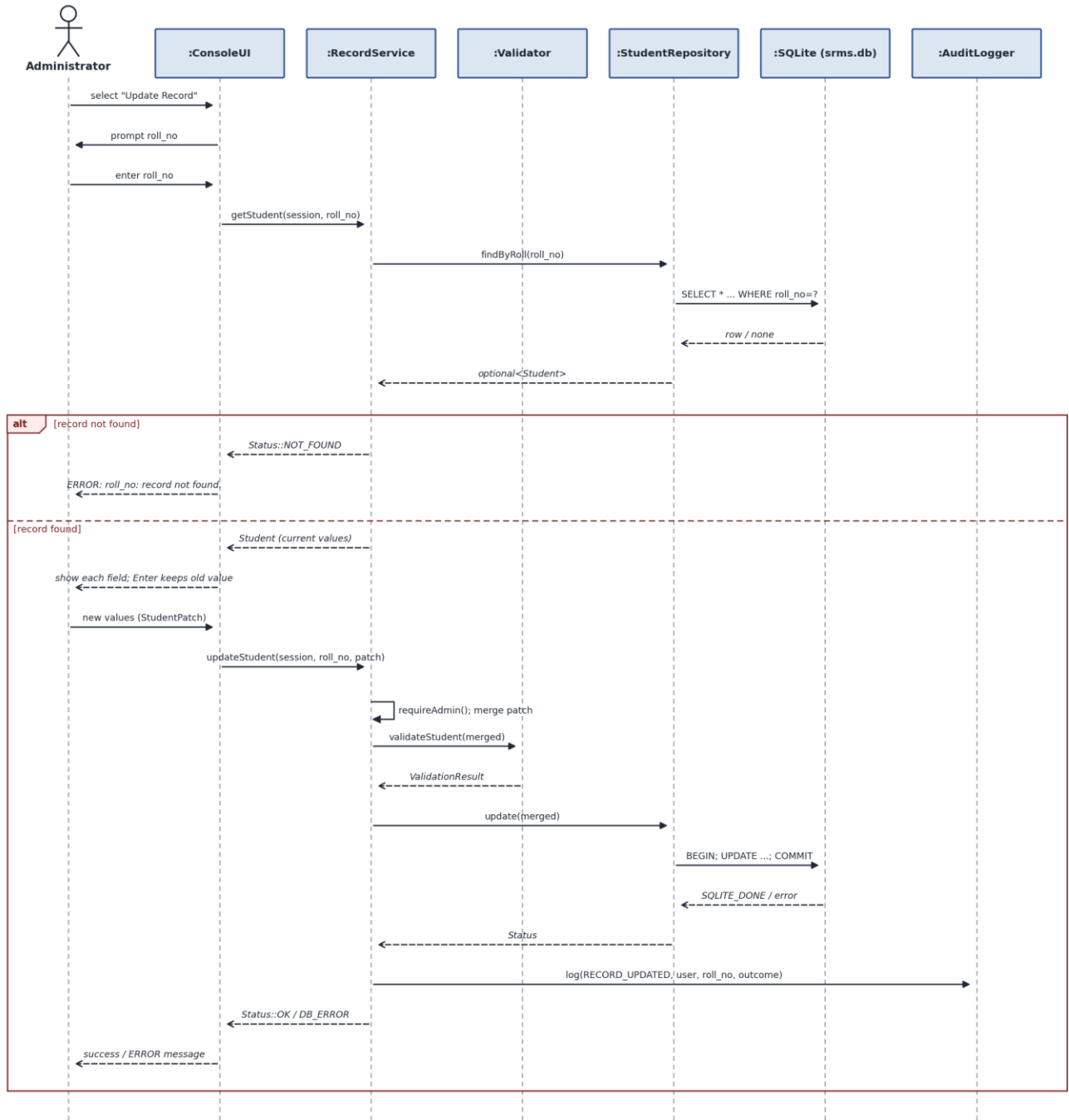


Figure 4.3 — Sequence Diagram: Update Student Record (UC-04, includes UC-11; REQ-CRUD-03, 04, 07, 08; REQ-DATA-01, 07; SEC-04, 07)

Only changed fields are sent as a StudentPatch (roll_no is immutable, BR-02); RecordService::updateStudent() merges the patch into the stored record and validates the merged result as a whole (catching cross-field rules, not just per-field ones) before StudentRepository::update() commits inside a transaction and logs RECORD_UPDATED.

4.2.4 Search Student Record (Owner: Chaithanya N)

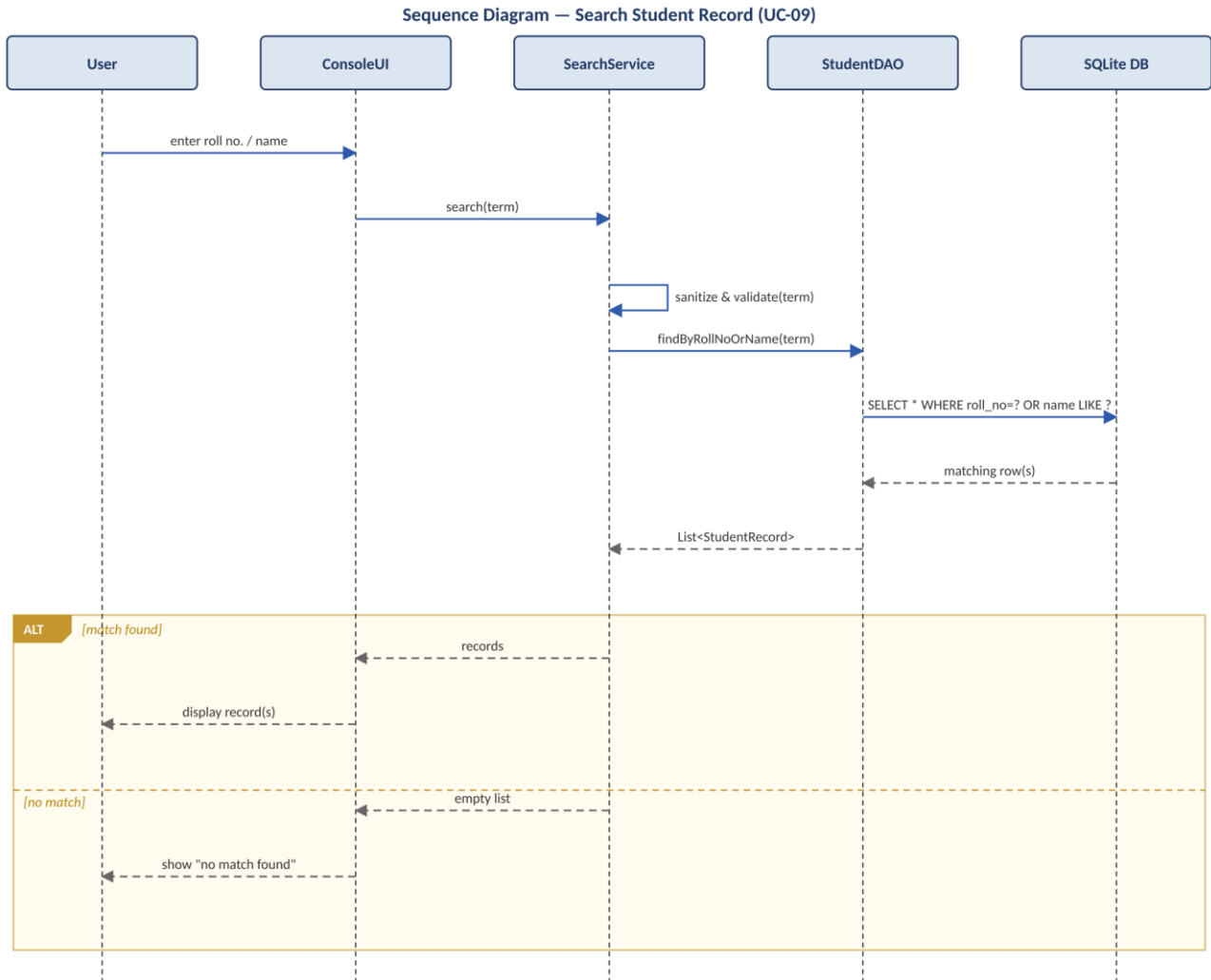


Figure 4.4 — Sequence Diagram: Search Student Record (UC-09; REQ-SRCH-01..04)

SearchService::search() sanitizes and validates the query term, then StudentDAO issues a single parameterized SELECT matching roll_no exactly or name by substring (both case-insensitive, SEC-02). A zero-row result is a valid OK (empty Page), not an error — ConsoleUI displays “No matching record found.” (REQ-SRCH-03). For a Viewer session, SearchService strips contact and the timestamp fields from each Student before returning the Page (REQ-SRCH-04, SEC-09).

4.2.5 Sort / Filter Records (Owner: Chaithanya N)

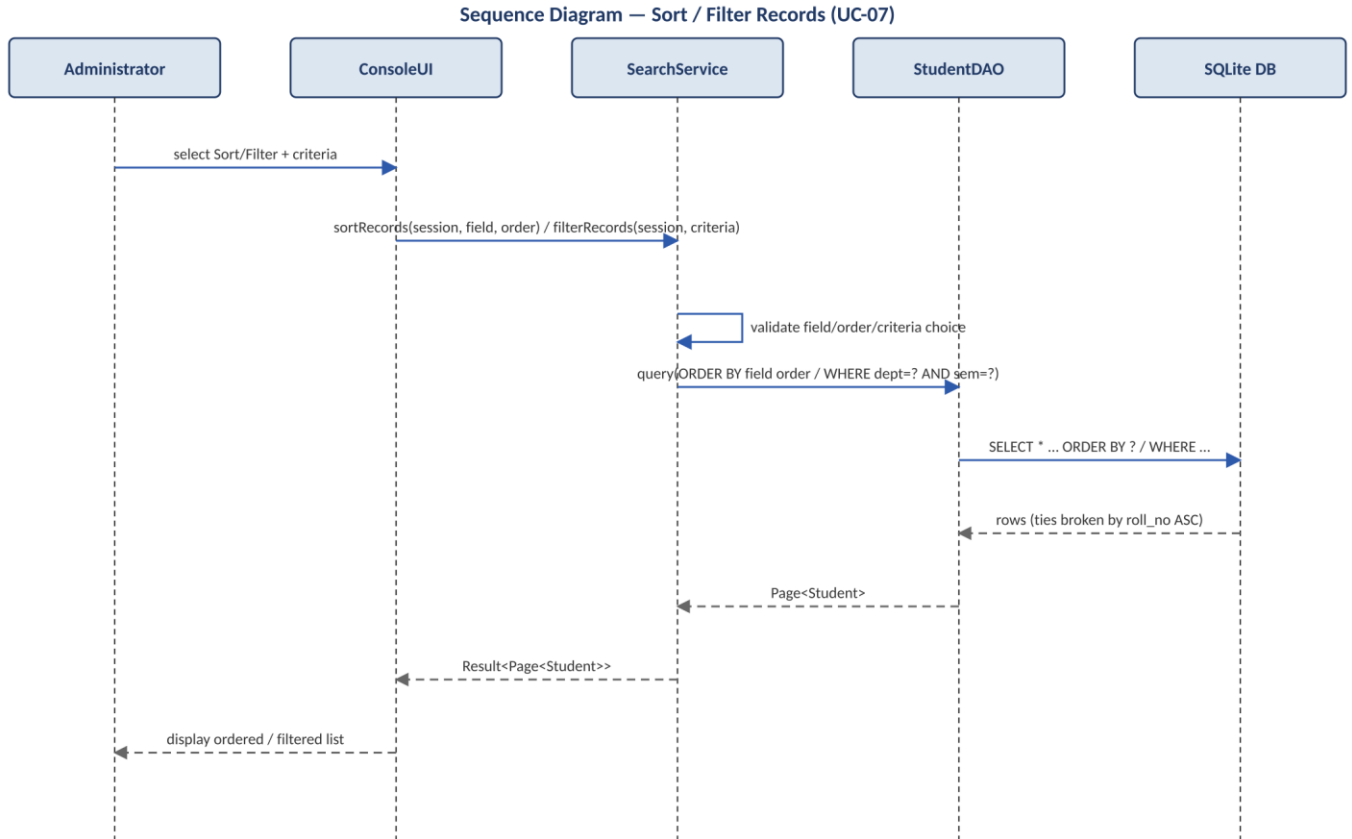


Figure 4.5 — Sequence Diagram: Sort / Filter Records (UC-07; REQ-SRCH-05..07)

`sortRecords()` and `filterRecords()` share the same `StudentDAO` query path: the chosen field/order or department/semester criteria are validated against a fixed allow-list before being used to build a parameterized `ORDER BY / WHERE` clause (never string-concatenated, SEC-02). Sort ties are always broken by `roll_no` ascending (REQ-SRCH-06).

4.2.6 Generate & Export Report (Owner: Arabhi Bhat)

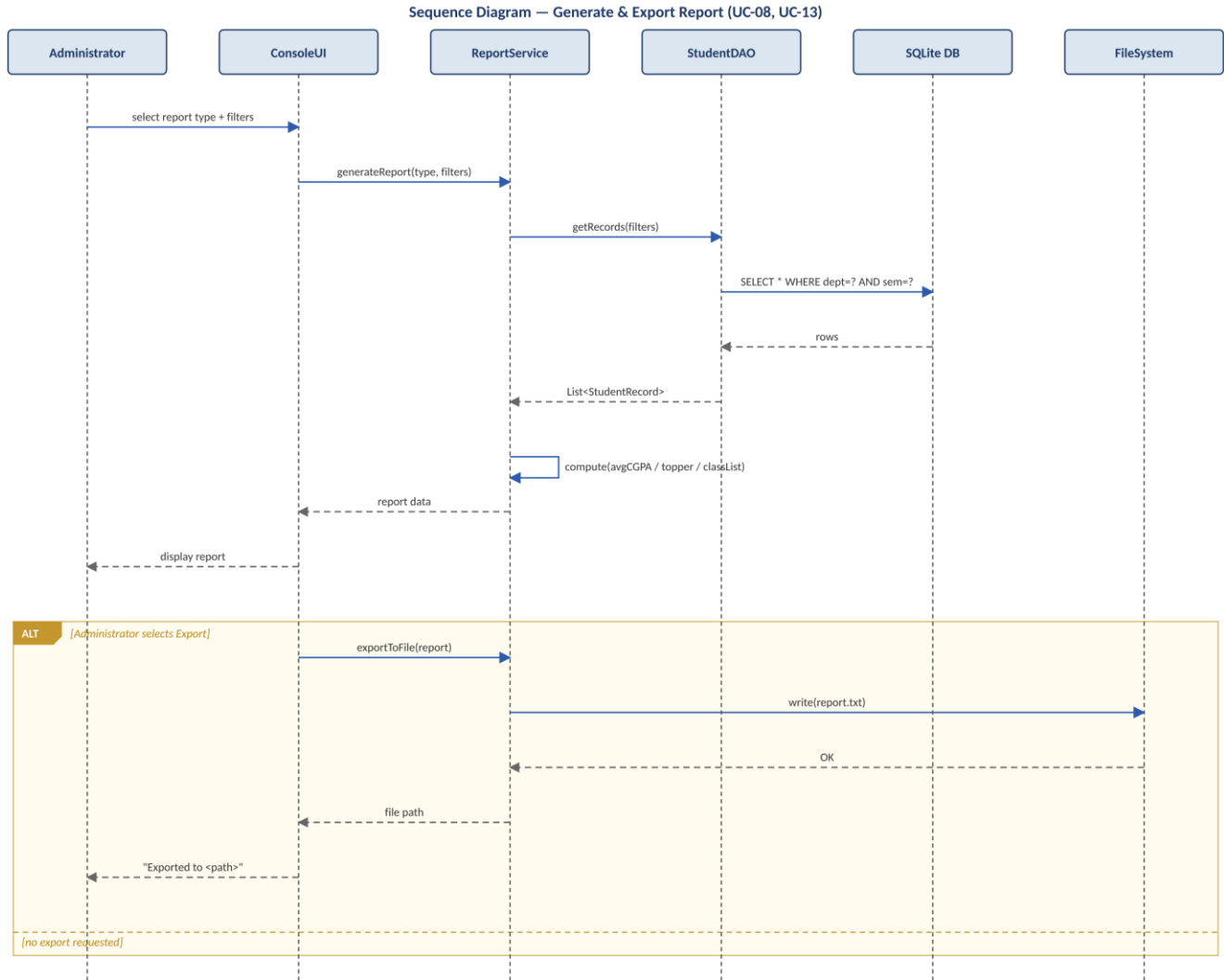


Figure 4.6 — Sequence Diagram: Generate & Export Report (UC-08, UC-13; REQ-RPT-01..04)

4.3 API Design

SRMS is a single-process console application with no network layer, so ‘API’ here means the in-process module interface (function signatures) between a Controller and its Service — not a REST/HTTP API. Interfaces are given for all four owned components (exceeding the minimum of two).

4.3.1 Shared Types (src/core/types.h)

Defined by Bhavyaa Garg as part of the CRUD component design; adopted as the single canonical type set for every module so that Auth, Search and Reports return the same Status/Result<T> shapes rather than each inventing its own error handling.

- enum class Status { OK, VALIDATION_ERROR, DUPLICATE_KEY, NOT_FOUND, ACCESS_DENIED, ACCOUNT_LOCKED, CANCELLED, DB_ERROR, IO_ERROR, INTERNAL_ERROR };
- struct Session { std::string username; bool is_admin; std::time_t last_activity; };

- struct Student { roll_no, name, department, semester, contact, cgpa, created_at, updated_at; } — field rules per SRS Appendix B.
- struct StudentPatch { optional<...> name, department, contact, semester, cgpa; } — only changed fields are set (used by RecordService::updateStudent).
- template<typename T> struct Result { Status status; T value; std::string detail; }; and
template<typename T> struct Page { vector<T> items; int page, total_pages, total_items; };

4.3.2 Per-Component Interfaces

Component	Interface (signature)	Errors / Notes
AuthService	Result<Session> login(const string& username, const string& password) void logout(Session&) bool hasRole(const Session&, Role)	Status::ACCESS_DENIED, ACCOUNT_LOCKED (SEC-03) — same shared Status enum as CRUD
RecordService (Bhavyaa Garg)	Status addStudent(const Session&, const Student&) Result<Student> getStudent(const Session&, const string& roll_no) Status updateStudent(const Session&, const string& roll_no, const StudentPatch&) Status deleteStudent(const Session&, const string& roll_no)	Status::VALIDATION_ERROR, DUPLICATE_KEY, NOT_FOUND, ACCESS_DENIED — see Section 4.2.2–4.2.3
SearchService (Chaithanya N)	Result<Page<Student>> search(const optional<Session>&, const string& term) Result<Page<Student>> sortRecords(const Session&, Field, Order) Result<Page<Student>> filterRecords(const Session&, const FilterCriteria&)	Field visibility depends on role (SEC-09, uses shared Session); an empty Page is a valid OK result, not an error
ReportService (Arabhi Bhat)	Result<Report> generate(const Session&, ReportType, const Filters&) Result<string> exportToFile(const Session&, const Report&, Format)	Status::IO_ERROR on export failure; empty dataset is OK with zero rows (REQ-RPT-05)

4.4 Error Handling, Logging & Monitoring

Lead: Bhavyaa Garg. CI/monitoring angle: Arabhi Bhat.

4.4.1 Error-Handling Strategy

- Detect early: input errors are caught by Validator at entry, before any Service or database call (REQ-DATA-02..05).
- Return, don't throw: errors travel upward as Status values (Section 4.3.1). Each layer either handles an error or passes it up unchanged; no layer swallows one.
- Translate once: only ConsoleUI turns a Status into text, via ErrorCatalog::message(), always in the format “ERROR: <field>: <rule>.” (UI-03). Internal details (SQLite codes, file paths) are logged but never shown to the user.

- Fail safe: every write is a single transaction; any failure triggers ROLLBACK, so the database is never left half-written (NFR-REL-01).
- Never crash on user input: the only fatal errors are the start-up conditions below; after start-up every error returns the user to the previous menu (NFR-REL-02).
- Deny by default: a missing or expired session always yields ACCESS_DENIED, never partial access.

4.4.2 Error Catalogue

Status	Raised by	User message (console)	Log level
VALIDATION_ERROR	Validator	ERROR: <field>: <rule>.	WARN
DUPLICATE_KEY	RecordService	ERROR: roll_no: <value> already exists.	WARN
NOT_FOUND	RecordService	ERROR: roll_no: record not found.	INFO
ACCESS_DENIED	All services	ERROR: access: administrator login required.	WARN + audit
ACCOUNT_LOCKED	AuthService	ERROR: login: account locked, try again in <n>s.	WARN + audit
CANCELLED	ConsoleUI	Operation cancelled. / Deletion cancelled.	INFO
DB_ERROR	Repository	ERROR: database: <operation> failed.	ERROR
IO_ERROR	ReportService	ERROR: file: could not write <file name>.	ERROR
INTERNAL_ERROR	Service boundary catch	ERROR: internal: unexpected error, see srms.log.	ERROR

4.4.3 Start-up / Fatal Conditions

Condition at start-up	Action	Exit code
srms.db missing	Create it with schema.sql (REQ-DATA-08); continue	—
srms.db cannot be opened (permissions, locked)	Print “FATAL: cannot open database srms.db.”, log, exit	2
PRAGMA integrity_check does not return “ok”	Print “FATAL: database integrity check failed.”, log, exit without writing	3
srms.log cannot be opened	Warn and continue, logging to stderr only	—

4.4.4 Transaction & Recovery Rules

- Every insert/update/delete uses BEGIN IMMEDIATE ... COMMIT; any failure between the two issues ROLLBACK before returning DB_ERROR.
- SQLite runs with PRAGMA journal_mode=WAL and PRAGMA foreign_keys=ON, so an interrupted write is never visible after a crash (NFR-REL-01).
- SQLITE_BUSY is retried via sqlite3_busy_timeout(db, 2000); after 2 s it becomes DB_ERROR.
- Every sqlite3_stmt is finalised on every path via an RAII wrapper (StmtGuard), so nothing leaks on early returns (NFR-REL-03).

4.4.5 Logging & Monitoring

SRMS keeps two separate logs with different purposes: an application log (srms.log, written by Logger across all modules, for diagnosing failures) and the audit_log table in srms.db (written only by AuditLogger from within Services, for security accountability per SEC-04). Passwords, hashes and salts are never written to either (SEC-05). Monitoring in this version is CI- and log-based rather than live: the GitHub Actions build/test pass rate is the team's health signal (NFR-TST-02), and audit_log is queryable via the sqlite3 CLI for post-hoc review.

4.5 UX Design

Console UX follows the numbered-menu design in SRS Section 3.1: every screen shows a title line (UI-01); an out-of-range or non-numeric menu choice re-displays the menu rather than crashing (UI-02); tabular output (record lists, reports) is kept within an 80-character terminal width, truncating long names with "..." (UI-04); and every operation returns to the previous menu on completion or cancellation (UI-05). Password entry is never echoed to the console (REQ-AUTH-03). Owner: B Varun (menu/session owner), consumed by all four Controllers.

4.6 Open Issues & Next Steps

- Decide whether CSV export (REQ-RPT-04) uses a hand-rolled writer or a small header-only CSV library — to be resolved in Sprint 5.
- Confirm the exact buffer-size constants enforcing SEC-10 for each field in Appendix B, and add them to a shared constants header.
- Spike needed in Sprint 1: verify the OpenSSL libcrypto build works unchanged on Windows/MinGW-w64 (NFR-PORT-01) before other modules depend on password hashing.
- Future enhancement (explicitly out of v1.0 scope): an optional GUI or web front-end, per the Synopsis 'Future Enhancements' section — the layered design is intended to make this additive rather than a rewrite.

5. Appendices

5.1 Glossary

See SRS Appendix A for product-level terms. Architecture-specific terms are defined in Section 1.4 of this document (DAO, Service, Controller, STRIDE).

5.2 References

- Software Requirements Specification (SRS), SRMS, Version 2.0, Team 17.
- Team 17 — SRMS Synopsis / Project Proposal, Version 1.0.
- OWASP Foundation — “Password Storage Cheat Sheet” and “SQL Injection Prevention Cheat Sheet.”
- OMG Unified Modeling Language (UML) Specification, Version 2.5.1.

5.3 Tools

- Diagramming: diagrams in this document were generated programmatically to an SVG/UML sequence-diagram convention; Draw.io/PlantUML are acceptable alternatives for future revisions.
- GoogleTest (unit testing), cppcheck (static analysis), GitHub Actions (CI).

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Test Plan (STP) — Student Record Management System (SRMS)

Project: Student Record Management System (SRMS)

Version: 1.0

Authors: Team 17

B Varun (PES1UG24CS109), Bhavyaa Garg (PES1UG24CS116)

Chaithanya N (PES1UG24CS124), Arabhi Bhat (PES1UG24CS927), Team Lead

Date: 1 October 2026

Status: Draft

1. Introduction

Purpose: This document defines the test plan for SRMS v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing.

Scope: Testing covers Administrator Authentication, Student Record CRUD, Search/Sort/Filter, Reports & Statistics, Data Persistence & Validation, the console UI, and the security requirements in SRS Section 6.3. Hardware testing is excluded (SRMS has no custom hardware).

References: SRMS Software Requirements Specification (SRS) v2.0, SRMS Software Architecture and Design Specification (SAD) v1.0, Team 17 Synopsis v1.0.

Definitions: SRMS (Student Record Management System), FR/NFR (Functional/Non-Functional Requirement), RTM (Requirements Traceability Matrix), CI (Continuous Integration), SRS (Software Requirements Specification).

2. Test Items

- Administrator Authentication & Access Control module (login, logout, lockout, session/role checks)
- Student Record CRUD module (add, update, delete, list)
- Search, Sort and Filter module
- Reports & Statistics module (class list, average CGPA, topper list, export)
- Data Persistence & Validation layer (SQLite schema, field validation, transactions)
- Console UI (menus, input handling, error-message formatting)
- Security controls (password hashing, parameterized queries, lockout, audit logging)

3. Features to be Tested

Features mapped to SRS requirement IDs:

- REQ-AUTH-01..08, SEC-01, 03, 04, 06, 07, 08: Administrator Authentication & Access Control (Owner: B Varun)
- REQ-CRUD-01..08: Student Record CRUD (Owner: Bhavyaa Garg)
- REQ-SRCH-01..07, SEC-09: Search, Sort & Filter (Owner: Chaithanya N)
- REQ-RPT-01..05: Reports & Statistics (Owner: Arabhi Bhat)
- REQ-DATA-01..08, SEC-02, 10: Data Persistence & Validation (Shared)
- UI-01..05: Console UI (Owner: B Varun)
- NFR-PERF-01..06, NFR-REL-01..04, NFR-USE-01/02, NFR-MNT-01..03, NFR-TST-01/02, NFR-PORT-01: Non-functional and quality attributes

4. Features Not to be Tested

- A GUI or web front-end — out of scope for v1.0 (console-only per Synopsis Scope table)
- Integration with external portals, government databases or a real bank/ERP system
- Biometric or hardware-based attendance capture — no such hardware exists in this project
- Cloud or multi-campus deployment and concurrent multi-Administrator writes (SRS Section 2.7 assumption)
- Email/SMS notification delivery — not a feature of SRMS

5. Test Approach / Strategy

Levels:

- Unit tests (GoogleTest, module-level) — one suite per Service/DAO, ≥70% line coverage of src/core/ (NFR-TST-01)
- Integration tests — Service ↔ DAO ↔ SQLite, verifying transactions, constraints and rollback behaviour
- System tests — end-to-end console walkthroughs of each use case (UC-01 to UC-13)
- Acceptance tests — final demo walkthrough against the Synopsis objectives and SRS fit criteria

Types:

- Functional testing (every FR in SRS Section 5)
- Negative / boundary testing (invalid input, duplicate roll numbers, out-of-range CGPA/semester)
- Regression testing (full suite re-run on every CI push, NFR-TST-02)
- Performance testing (NFR-PERF-01..06 timing targets at 10,000-record scale)
- Security testing (see Section 5.1)
- Usability and portability testing (NFR-USE-01/02 novice-user walkthrough; NFR-PORT-01 dual-OS build)

Entry Criteria: a build that compiles cleanly with -Wall -Wextra and 0 cppcheck high-severity findings (NFR-MNT-03); schema.sql applied; test data loaded.

Exit Criteria: all planned test cases in Section 13 executed; 0 critical (High-priority) defects open; every REQ/NFR/SEC ID in the RTM shows a Pass result or an accepted, documented exception.

5.1 Security Validation

- Validate password storage: PBKDF2-HMAC-SHA256 hashing with unique per-account salts, no plaintext (SEC-01)
- SQL-injection fuzzing on every free-text field (search, add, update) using payloads such as ' OR '1'='1 and ; DROP TABLE students;-- (SEC-02)
- Brute-force lockout verification: account locks for 300s after 5 consecutive failed logins (SEC-03)
- Audit-log verification for all required event types — login, lockout, add, update, delete, report export (SEC-04)
- Confirm no password, hash or salt value ever appears in console output, logs or exported files (SEC-05)
- Forced default-password-change verification before the Admin Menu becomes reachable (SEC-06)
- Service-layer authorization check: all 6 modifying service functions reject calls without an active session (SEC-07)
- File-permission check: srms.db and the audit log are created with owner-only 0600 permissions on Linux (SEC-08)
- Verify Viewer search results exclude restricted fields (contact, timestamps) (SEC-09)
- Long-input fuzzing (up to 300 characters) across every console prompt for buffer-safety, cross-checked with cppcheck (SEC-10)
- Security Objective Coverage check: confirm Confidentiality, Integrity, Authentication & Authorisation, Accountability, and Availability & Resilience (SO-01..SO-05) are each covered by at least one passing case

6. Test Environment

Hardware: reference machine — 4-core x86-64 CPU, 8 GB RAM, SSD (matches SRS Section 6.1 performance baseline).

Software: Ubuntu 22.04 LTS (primary) and Windows 11 with MinGW-w64 (secondary); SQLite 3.31+; OpenSSL libcrypto 1.1.1+; C++17 toolchain.

Tools: GoogleTest (unit/integration), cppcheck (static analysis), Valgrind (memory-leak checks), lcov/gcov (coverage), GitHub Actions (CI), sqlite3 CLI (direct DB inspection for security tests), a fuzzing script for long-input/injection cases.

Test Data: a synthetic dataset of 10,000 student records (performance baseline) plus small hand-crafted datasets (3–45 records) for functional and boundary tests; 2–3 seeded Administrator accounts.

Representative boundary values:

Field	Valid Examples	Invalid Examples
roll_no	PES1UG24CS116; ABCDE; A12345	AB1; PES-116; blank; >20 chars
name	A; Anne-Marie O'Neil; 80-char valid name	blank; 81 chars; R2D2; unsupported symbols
department	CSE; ECE; EEE; ME; CV; BT; AIML	XYZ; blank
semester	1; 5; 8	0; 9; 2.5; blank
contact	9876543210; test@example.com	12345; malformed email; >50 chars
cgpa	0.00; 8.41; 10.00	-0.01; 10.01; 8.123; blank
delete confirmation	YES	yes; Y; NO; blank

7. Test Schedule

Milestones:

- Test case design (this document) — Day 1 of Deliverables Part-1 window
- Unit tests written alongside each Sprint's feature (Sprints 1–5) — ongoing, per Synopsis Plan of Work
- Integration + system test execution — Sprint 5–6
- Security validation pass (Section 5.1) — Sprint 6, before final validation
- Performance test run (10,000-record dataset) — Sprint 6
- Final Test Summary Report — end of Sprint 6 / final submission

8. Test Deliverables

- This Test Plan
- Test cases (Section 13) and their GoogleTest implementations
- Unit test coverage report (gcov/lcov, NFR-TST-01)
- cppcheck static-analysis report
- CI execution logs (GitHub Actions)
- Security validation results (Section 5.1, completed)
- Defect log and Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
Primary Test Lead	B Varun	Author this Test Plan; own Section 5 strategy and Section 5.1 security validation; design & execute TC-AUTH-*, TC-UI-*, TC-SEC-* test cases.
Test Engineer — CRUD	Bhavyaa Garg	Design & execute TC-CRUD-*, TC-DATA-* test cases; report defects.
Test Engineer — Search	Chaithanya N	Design & execute TC-SRCH-* test cases; report defects.
Test Engineer — Reports	Arabhi Bhat	Design & execute TC-RPT-*, TC-PERF-* test cases; maintain CI pipeline.
Product Owner / Sign-off	Arabhi Bhat	Approve test results and readiness for final submission.

10. Risks and Mitigation

Risk	Mitigation
Two-day deliverable window leaves little slack for test execution	Unit tests written alongside each feature, not deferred; CI catches regressions immediately on every push.
OpenSSL/MinGW build issues on Windows block security tests	Confirm the build early on Windows; Ubuntu remains the primary CI target if MinGW blocks.
10,000-record performance dataset not ready in time	Fall back to a smaller (1,000-record) dataset for an initial pass; re-run at full scale before final submission.
Team members' test cases diverge in style/format	All test cases follow the single convention and table format in Section 13.

11. Assumptions & Dependencies

- The GitHub Actions CI environment remains available throughout the test window
- Each module owner has their feature code-complete enough to unit-test before Sprint 6
- The reference evaluation machine (SRS Section 6.1) is representative of the course's grading environment
- Test data generation scripts (for the 10,000-record dataset) are shared in the repository, not hand-typed by each tester

12. Suspension & Resumption Criteria

Suspend testing if:

- Environment unavailable for >4 hours
- A defect blocks more than 30% of planned test cases from running

Resume testing if:

- The blocking build/defect is fixed
- A green CI run is confirmed on the affected branch

13. Test Case Management & Traceability

The RTM ensures mapping of SRS requirements to test cases. Test-case IDs match the ranges reserved in SRS Appendix C. The 44 cases below (well above the course's 7–10 minimum) cover every functional, UI, performance, reliability, security and quality requirement in the SRS.

Test Cases:

Functional and UI:

Test ID	Type	Pri.	Description / Steps	Req ID(s)	Expected Result
TC-AUTH-01	Functional	High	Login with correct username & password on a clean DB with a valid Admin account.	REQ-AUTH-01, 06	Admin Menu opens; unauthorized direct service calls return ACCESS_DENIED.
TC-AUTH-02	Functional	High	Try wrong username/correct-format password, then correct username/wrong password; observe echo.	REQ-AUTH-01..03	Both failures show identical error text; password characters are never echoed.
TC-AUTH-03	Security	High	5 consecutive invalid logins; retry immediately; retry after 300s.	REQ-AUTH-04, SEC-03	Account locked 300s; immediate retry refused; login succeeds after lock period.
TC-AUTH-04	Functional	Medium	Logout and observe Main Menu; start new session and leave idle.	REQ-AUTH-07, 08	Logout ends session immediately; inactivity ends session between 600–610s.
TC-AUTH-05	Security	High	Attempt Add/Update/Delete/List/Sort-Filter/Report via UI and service calls with no session.	REQ-AUTH-05, SEC-07	Viewer can only Search; all six modifying/admin services reject unauthenticated access.
TC-AUTH-06	Security	High	Login with default password; try to skip change; try non-compliant new passwords; set compliant one.	SEC-06	Admin Menu unreachable before change; non-compliant passwords rejected; compliant one accepted.
TC-CRUD-01	Functional	High	Add Record with valid, unique roll_no and all required fields.	REQ-CRUD-01, REQ-DATA-02..05	Record stored and retrievable with the six entered values.
TC-CRUD-02	Functional	High	Add Record re-using an existing roll_no.	REQ-CRUD-02, REQ-DATA-06	Duplicate rejected with the specified roll_no error; record count unchanged.
TC-CRUD-03	Functional	High	Update each editable field singly; repeat pressing Enter to keep value; restart app.	REQ-CRUD-03, 04, 08	Fields change; Enter keeps existing value; roll_no unchanged; updated_at changes and persists.

Test ID	Type	Pri.	Description / Steps	Req ID(s)	Expected Result
TC-CRUD-04	Functional	High	Delete Record; try blank, NO, yes, Y at confirmation; then exact YES.	REQ-CRUD-05, 07, NFR-SAFE-01	All non-exact inputs cancel deletion; exact YES deletes and logs the event.
TC-CRUD-05	Functional	Medium	View All Records with 45 valid records seeded; check pages/rows/line width.	REQ-CRUD-06, UI-04, UI-05	3 pages shown as 20/20/5, ascending roll_no; output fits 80 cols; returns to prior menu.
TC-SRCH-01	Functional	High	Search for an existing roll_no in lower-case, and for a nonexistent roll_no.	REQ-SRCH-01, 03	Case-insensitive exact match returns the record; nonexistent search shows the required message.
TC-SRCH-02	Functional	Medium	Search name with 1 character, then “var”, then unmatched text.	REQ-SRCH-02, 03	1-char query rejected; 2+ char query returns every containing match by roll_no; empty shows message.
TC-SRCH-03	Security	High	Search a record as Viewer; capture displayed fields.	REQ-SRCH-04, SEC-09, REQ-AUTH-05	Viewer sees only roll_no, name, department, semester, cgpa; contact/timestamps absent.
TC-SRCH-04	Functional	High	Sort each field ascending/descending; check equal-key order; filter by dept/semester/both.	REQ-SRCH-05, 06, 07	All 6 sort combinations correct; ties use ascending roll_no; combined filter satisfies every criterion.
TC-RPT-01	Functional	Medium	Generate Class List for a populated selection, then an empty one.	REQ-RPT-01, 05	Populated report matches SQL row count/order; empty selection shows message and creates no file.
TC-RPT-02	Functional	Medium	Generate Average CGPA report with known CGPA values; compare to hand-computed mean.	REQ-RPT-02	Average equals hand-computed value ± 0.005 , shown to 2 decimal places.
TC-RPT-03	Functional	Medium	Generate Topper List with 2 tied students; export as TXT then CSV; inspect audit log.	REQ-RPT-03, 04, SEC-04	All tied toppers appear; files exist and match screen; REPORT_EXPORTED event recorded.
TC-DATA-01	Functional	High	Delete/rename srms.db; start app; add a record; restart.	REQ-DATA-01, 08	Full schema auto-created; successful changes present after restart.
TC-DATA-02	Functional	High	Enter boundary-valid and invalid values for roll_no, name, semester, cgpa, department.	REQ-DATA-02..05	Boundary-valid values accepted; invalid values rejected with field-

Test ID	Type	Pri.	Description / Steps	Req ID(s)	Expected Result
					specific errors, only that field re-prompted.
TC-DATA-03	Functional	Medium	Force a write operation to fail (read-only DB); observe error handling.	REQ-DATA-07	Required DB error format shown; transaction rolled back; app returns to menu and keeps running.
TC-DATA-04	Security	High	Attempt direct duplicate INSERT via sqlite3 CLI; search/add using SQL injection payloads.	REQ-DATA-06, SEC-02	Duplicate INSERT fails on primary key; injection payloads treated as literal data; tables intact.
TC-UI-01	UI	High	Visit every menu; enter abc, -1, 99 where a numeric choice is expected.	UI-01, 02	Every screen has a title line; invalid/non-numeric choices do not crash and redisplay the menu.
TC-UI-02	UI	High	Trigger representative validation errors for roll_no, name, department, semester, cgpa.	UI-03, NFR-USE-02	Every error follows "ERROR: <field>: <rule violated>." and names the field/rule.
TC-UI-03	UI	High	View table/report output with a long name present; perform operation + cancellation paths.	UI-04, 05	No output line exceeds 80 chars; long names truncated with ...; operations return to prior menu.

Non-Functional, Reliability, Security and Quality:

Test ID	Type	Pri.	Description / Steps	Req ID(s)	Expected Result
TC-PERF-01	Performance	High	Run roll_no search 20 times on a 10,000-record DB on the reference machine; record mean/max.	NFR-PERF-01	Mean $\leq 0.5s$ and max $\leq 1s$.
TC-PERF-02	Performance	High	Run name search, sort, filter and each report type 20 times on the 10,000-record DB.	NFR-PERF-02, 03	Each search/sort/filter mean $\leq 2s$; each report mean $\leq 2s$.
TC-PERF-03	Performance	High	Measure Add/Update/Delete commit times, startup time, and peak RSS using defined tools.	NFR-PERF-04, 05, 06	CRUD commit mean $\leq 1s$; startup mean $\leq 3s$; peak RSS $\leq 50MB$.
TC-REL-01	Reliability	High	Force termination during 20 write operations; run PRAGMA integrity_check after each.	NFR-REL-01	integrity_check returns "ok" every time; DB is in the before- or after-state, never partial.
TC-REL-02	Reliability/Security	High	Send 1,000 randomized inputs across all menus, including over-long strings.	NFR-REL-02, SEC-10	0 abnormal crashes; over-limit input rejected before unsafe copy; no truncation-

Test ID	Type	Pri.	Description / Steps	Req ID(s)	Expected Result
					induced corruption.
TC-REL-03	Reliability	Medium	Run 500 mixed operations twice on a seeded dataset; inspect Valgrind output.	NFR-REL-03, 04	0 errors; 0 bytes definitely lost; repeated output byte-identical except timestamps.
TC-SEC-01	Security	High	Inspect admins table; compare salts/hashes for two accounts with equal passwords.	SEC-01	No plaintext password stored; PBKDF2-HMAC-SHA256 with unique 16-byte salts, ≥100,000 iterations.
TC-SEC-02	Security	High	Use payloads ' OR '1'='1 and ; DROP TABLE students;-- in add/search/update fields; inspect SQL.	SEC-02	No string-concatenated user input; payloads treated literally; all tables remain intact.
TC-SEC-03	Security	High	Execute one successful login, failed login, lockout, add, update, delete, and report export.	SEC-04	Exactly 7 audit events exist with timestamp, username, event type, target_roll_no, outcome.
TC-SEC-04	Security	High	Inspect console output, security log and report exports after a full scripted session.	SEC-05	0 matches for password, password hash, or salt values.
TC-SEC-05	Security	Medium	Check file permissions for srms.db and audit log using ls -l on first run (Linux).	SEC-08	Both files show owner-only 0600 permissions.
TC-SEC-06	Security	High	Send 300-character input to every prompt; inspect behaviour plus cppcheck.	SEC-10	No crash, no unsafe truncation, 0 buffer-overflow warnings from cppcheck.
TC-SEC-07	Security	High	Invoke each of the 6 modifying service functions without an active Admin session.	SEC-07	All six return ACCESS_DENIED before performing any database change.
TC-SEC-08	Security	High	Search one record as Viewer; inspect the complete output.	SEC-09	Only roll_no, name, department, semester, cgpa appear for the result.
TC-SEC-09	Security	High	Execute the grouped confidentiality, integrity, role, audit and resilience scenarios together.	SO-01..05	Observed controls collectively satisfy the measurable SEC-01..10 fit criteria.

Test ID	Type	Pri.	Description / Steps	Req ID(s)	Expected Result
TC-QA-01	Usability	Medium	Ask 5 novice participants to add one record using only on-screen prompts, no help.	NFR-USE-01	At least 4 of 5 finish within 5 menu interactions and 2 minutes.
TC-QA-02	Analysis	Medium	Inspect module separation; run cppcheck; compile with -Wall -Wextra.	NFR-MNT-01..03	No src/core console I/O; 0 compiler warnings; 0 high-severity cppcheck findings.
TC-QA-03	Analysis	Medium	Review coverage and CI history for recent pushes.	NFR-TST-01, 02	src/core line coverage $\geq 70\%$; every push triggers build/tests, each run completes within 10 minutes.
TC-QA-04	Portability	Medium	Build and run the same source without modification on Ubuntu 22.04 and Windows 11/MinGW; smoke test.	NFR-PORT-01	Both environments build and pass the required tests without source changes.

Requirements Traceability Matrix (RTM):

- REQ-AUTH-01..08, SEC-01, 03, 04, 06, 07, 08 (Administrator Authentication & Access Control) → TC-AUTH-01..06, TC-SEC-01, 03, 04, 06, 07
- REQ-CRUD-01..08 (Student Record CRUD) → TC-CRUD-01..05, TC-DATA-01, 02
- REQ-SRCH-01..07, SEC-09 (Search, Sort & Filter) → TC-SRCH-01..04, TC-SEC-08
- REQ-RPT-01..05 (Reports & Statistics) → TC-RPT-01..03
- REQ-DATA-01..08, SEC-02, 10 (Data Persistence & Validation) → TC-DATA-01..04, TC-SEC-02, 06
- UI-01..05 (Console UI) → TC-UI-01..03, TC-CRUD-05, TC-SRCH-03
- NFR-PERF-01..06 (Performance) → TC-PERF-01..03
- NFR-REL-01..04, NFR-SAFE-01 (Reliability & Safety) → TC-REL-01..03, TC-CRUD-04
- SEC-01..10, SO-01..05 (Security, consolidated) → TC-SEC-01..09, TC-AUTH-03, 06, TC-REL-02
- NFR-USE-01/02, NFR-MNT-01..03, NFR-TST-01/02, NFR-PORT-01 (Quality Attributes) → TC-QA-01..04

Detailed Test Case Specifications (representative sample):

The six cases below follow the IEEE 829 test-case specification layout: two per module (Record CRUD, Search, Reports), one functional path plus one non-functional/security dimension each. The remaining 38 cases are specified at summary depth above and expanded to this same depth during Implementation (Deliverable 4).

TC-03 — Add a valid student record and verify it persists (SRS ref: TC-CRUD-01) — Owner: Bhavyaa Garg

Requirements traced	REQ-CRUD-01, REQ-CRUD-08, REQ-DATA-01, NFR-PERF-04, SEC-04
Type / Level	Functional + Non-functional (performance) / System test
Priority	High
Preconditions	1. srms.db deleted, so the program creates an empty database at start (REQ-DATA-08). 2. Test Administrator account admin / Test@1234 exists and is logged in. 3. Roll number PES1UG24CS999 does not exist.
Test data	roll_no = pes1ug24cs999 (lower-case, to check normalisation); name = Test Student; department = CSE; semester = 5; contact = 9876543210; cgpa = 8.75
Steps	1. From the Administrator Menu select 1 (Add Record). 2. Enter the test data values at each prompt. 3. Note the time from the last Enter to the confirmation message. 4. Select Logout, then Exit. 5. Restart the program; from the Main Menu select Search Record (Viewer) and enter PES1UG24CS999. 6. Run: <code>sqlite3 srms.db "SELECT created_at, updated_at FROM students WHERE roll_no='PES1UG24CS999';"</code> 7. Run: <code>sqlite3 srms.db "SELECT event_type, outcome FROM audit_log WHERE target_roll_no='PES1UG24CS999';"</code>
Expected results	Step 2: no error prompts. Step 3: "Record PES1UG24CS999 added successfully." shown within 1 second. Step 5: record shows PES1UG24CS999 Test Student CSE 5 8.75, no contact shown to Viewer. Step 6: created_at = updated_at, within 2 seconds of add time. Step 7: exactly one row, RECORD_ADDED OK.
Pass / Fail criteria	Pass only if every expected result matches. Any missing field, changed value, extra audit row, or confirmation taking over 1 second is a Fail.
Automated equivalent	RecordServiceTest.AddValidStudentPersistsAcrossReopen (GoogleTest; opens and closes the DB connection between add and read)
Actual result / Status	(to be filled during execution)

TC-04 — Delete requires the exact confirmation "YES" (SRS ref: TC-CRUD-02) — Owner: Bhavyaa Garg

Requirements traced	REQ-CRUD-05, REQ-CRUD-07, NFR-SAFE-01, SEC-04
Type / Level	Functional + Non-functional (safety) / System test
Priority	High
Preconditions	1. Administrator admin is logged in. 2. Record PES1UG24CS999 exists (created by TC-03 or seeded). 3. Record count N is noted: <code>sqlite3 srms.db "SELECT COUNT(*) FROM students;"</code>
Test data	Confirmation inputs, in order: "yes", "Y", (empty Enter), "NO", "YES"
Steps	1. Select 3 (Delete Record) and enter PES1UG24CS999. 2. At the confirmation prompt type yes. 3. Repeat steps 1–2 with Y, then an empty Enter, then NO. 4. After each attempt, check the record count. 5. Repeat step 1 and type YES. 6. Search for PES1UG24CS999. 7. Select Delete Record again and enter PES1UG24CS999. 8. Query <code>audit_log</code> for <code>target_roll_no='PES1UG24CS999'</code> and <code>event_type='RECORD_DELETED'</code> .
Expected results	Step 1: record displayed, then "Type YES to confirm deletion of PES1UG24CS999:". Steps 2–4: each attempt shows "Deletion cancelled.", count stays N. Step 5: "Record PES1UG24CS999 deleted." shown, count becomes N–1. Step 6: "No matching record found." Step 7: "ERROR: roll_no: record not found.", count stays N–1. Step 8: exactly one row.

Pass / Fail criteria	Pass only if the record survives all 4 non-“YES” inputs, is deleted by “YES”, and exactly one RECORD_DELETED audit row exists. Any deletion without “YES” is a critical Fail (safety).
Automated equivalent	ConsoleUITest.ConfirmDeletionAcceptsOnlyExactYES (parameterised over the 5 inputs) and RecordServiceTest.DeleteMissingRollReturnsNotFound
Actual result / Status	(to be filled during execution)

TC-05 — Exact roll_no search is case-insensitive; unknown roll_no shows the required message (SRS ref: TC-SRCH-01) — Owner: Chaithanya N

Requirements traced	REQ-SRCH-01, REQ-SRCH-03, NFR-PERF-01
Type / Level	Functional + Non-functional (performance) / System test
Priority	High
Preconditions	1. Record PES1UG24CS116 exists in the database. 2. No session required — Search is available to both Administrator and Viewer (REQ-AUTH-05).
Test data	Search term “pes1ug24cs116” (lower-case, existing); search term “PES1UG24CS999” (does not exist)
Steps	1. From the Main Menu select Search Record. 2. Enter pes1ug24cs116 and note the time from Enter to the result being displayed. 3. Repeat the search with PES1UG24CS999.
Expected results	Step 2: the exact record for PES1UG24CS116 is returned, with mean response time $\leq 0.5s$ (NFR-PERF-01); the match succeeds despite the case difference. Step 3: “No matching record found.” is shown; no record is listed.
Pass / Fail criteria	Pass only if the case-insensitive match returns exactly the one correct record and the non-existent roll_no shows the exact required message, both within the timing budget.
Automated equivalent	SearchServiceTest.ExactRollNoSearchIsCaseInsensitive and SearchServiceTest.UnknownRollNoReturnsEmptyPage
Actual result / Status	(to be filled during execution)

TC-06 — Viewer search results exclude restricted fields (SRS ref: TC-SRCH-03) — Owner: Chaithanya N

Requirements traced	REQ-SRCH-04, SEC-09
Type / Level	Security / System test
Priority	High
Preconditions	1. No Administrator session active (Viewer/unauthenticated context). 2. Record PES1UG24CS116 has contact and created_at/updated_at values stored.
Test data	Search term “PES1UG24CS116”
Steps	1. From the Main Menu (no login) select Search Record. 2. Enter PES1UG24CS116. 3. Capture every field shown in the result. 4. Repeat the same search after logging in as Administrator and compare output.
Expected results	Step 3: only roll_no, name, department, semester, cgpa are shown; contact, created_at, updated_at are absent. Step 4: the Administrator view additionally shows contact, created_at and updated_at for the same record.

Pass / Fail criteria	Pass only if the Viewer output contains exactly the five permitted fields and none of the restricted fields, for every record returned.
Automated equivalent	SearchServiceTest.ViewerResultExcludesRestrictedFields (asserts field-set equality against the permitted five)
Actual result / Status	(to be filled during execution)

TC-07 — Average CGPA report matches a hand-computed mean (SRS ref: TC-RPT-02) — Owner: Arabhi Bhat

Requirements traced	REQ-RPT-02, NFR-PERF-02
Type / Level	Functional + Non-functional (performance) / System test
Priority	Medium
Preconditions	1. Administrator is logged in. 2. Department CSE, semester 5 contains exactly 3 seeded records with cgpa = 7.20, 8.50, 9.10.
Test data	department = CSE, semester = 5
Steps	1. From the Administrator Menu select Reports > Average CGPA. 2. Enter department CSE and semester 5. 3. Note the time to display and the value shown. 4. Hand-compute the mean: $(7.20+8.50+9.10)/3 = 8.2667$.
Expected results	Step 3: the report displays within a 2-second mean (NFR-PERF-02). Step 4: the displayed average equals 8.27 (rounded to 2 d.p.) and is within ± 0.005 of the hand-computed mean.
Pass / Fail criteria	Pass only if the displayed value matches the hand-computed mean to the stated tolerance and the 2-second timing budget is met.
Automated equivalent	ReportServiceTest.AverageCgpaMatchesManualComputation
Actual result / Status	(to be filled during execution)

TC-08 — Topper List includes every tied student, and export matches the screen (SRS ref: TC-RPT-03) — Owner: Arabhi Bhat

Requirements traced	REQ-RPT-03, REQ-RPT-04, SEC-04
Type / Level	Functional / System test
Priority	Medium
Preconditions	1. Administrator is logged in. 2. Two students, PES1UG24CS201 and PES1UG24CS202, are both seeded with cgpa = 9.80 (the CSE/semester-5 dataset maximum).
Test data	department = CSE, semester = 5, export format = CSV
Steps	1. From the Administrator Menu select Reports > Topper List for CSE, semester 5. 2. Observe the students listed. 3. Select Export and choose CSV. 4. Open the exported file and compare its rows to the on-screen report. 5. Run: <code>sqlite3 srms.db "SELECT event_type, outcome FROM audit_log WHERE event_type='REPORT_EXPORTED' ORDER BY rowid DESC LIMIT 1;"</code>
Expected results	Step 2: both PES1UG24CS201 and PES1UG24CS202 are listed as toppers (no single-winner tie-break). Step 4: the exported file's rows and values match the on-screen report exactly. Step 5: exactly one new row, REPORT_EXPORTED OK.
Pass / Fail criteria	Pass only if every tied student appears, the export file matches the screen exactly, and the export is recorded in the audit log.

Automated equivalent	ReportServiceTest.TopperListIncludesAllTiedStudents and ReportServiceTest.ExportMatchesGeneratedReport
Actual result / Status	(to be filled during execution)

14. Test Metrics & Reporting

Metrics collected:

- % test cases executed
- % passed/failed
- Defect density per module
- Defect aging
- Requirement coverage (% of SRS requirement IDs with at least one Pass result in the RTM)

Reports:

- Daily/running execution status (mirrored on the GitHub Projects board)
- Final Test Summary Report, compiled for the Deliverables Part-1 submission

15. Approvals

Role	Name	Signature / Date
Primary Test Lead	B Varun	
Team Lead	Arabhi Bhat	
Course Coordinator		